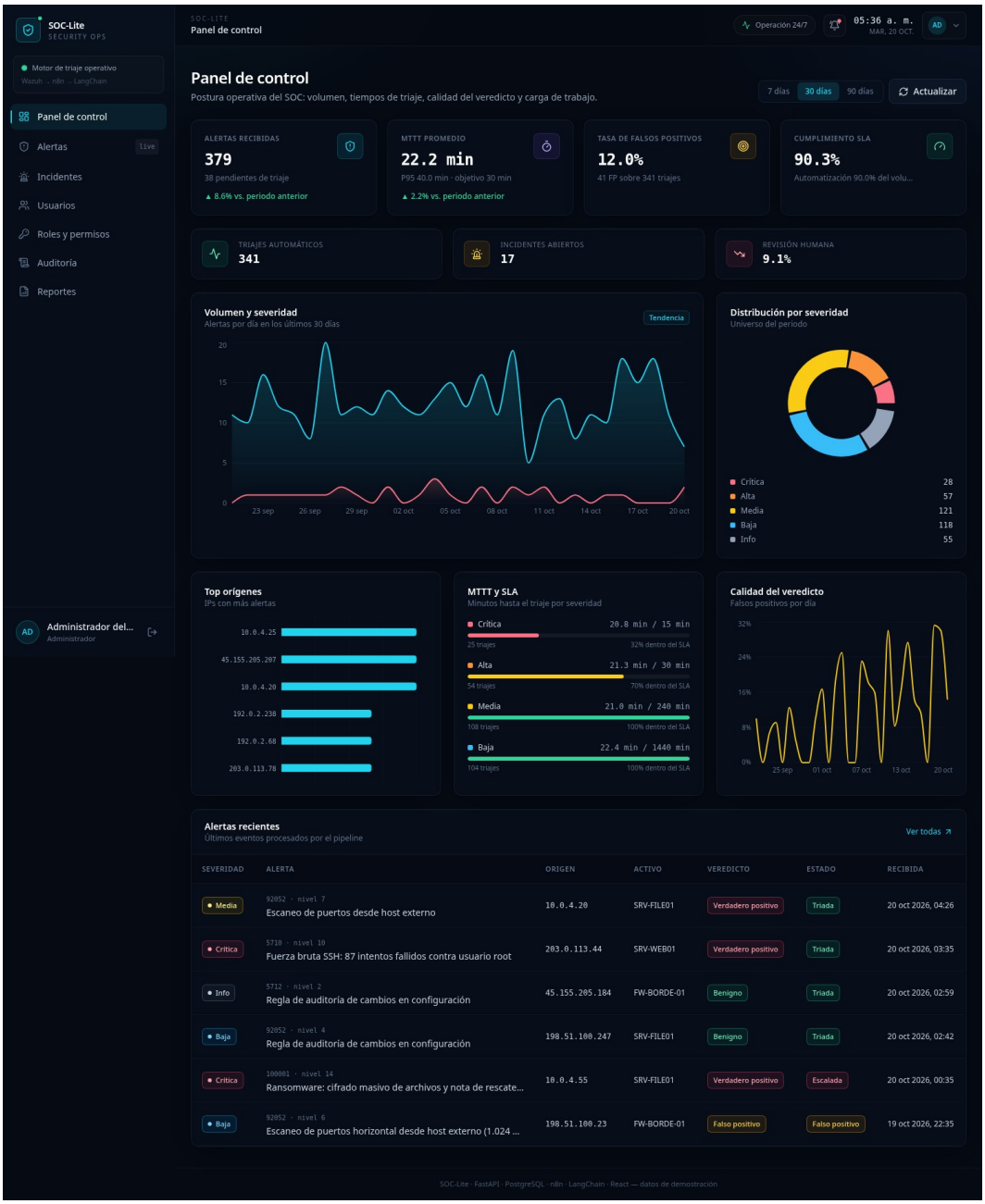
SOC-Lite

Triaje automático de alertas SOC

Documentación técnica, funcional y de operación



Caso de uso 1 (crítico) · FastAPI · PostgreSQL · n8n · LangChain/Ollama · React

PYME · mediana empresa · organismo público

Versión 1.0 · 20/10/2026

Contenido

1. Arquitectura del sistema (01-arquitectura.md)
2. Modelo de datos PostgreSQL y datos semilla (02-modelo-datos.md)
3. Desarrollo del backend (03-backend.md)
4. Desarrollo del frontend (04-frontend.md)
5. Flujo de nodos n8n (05-workflow-n8n.md)
6. Datasets públicos recomendados (06-datasets-publicos.md)
7. Stack tecnológico detallado (07-stack-tecnologico.md)

A. Contrato de API v1 (00-contrato-api.md)

Ficha del entregable

Esta documentación acompaña a una implementación funcional: esquema PostgreSQL, backend FastAPI, microservicio LangChain, frontend React, workflow n8n, pruebas y capturas de la aplicación en ejecución. Cada afirmación técnica se contrastó con el código o con una ejecución real; las limitaciones se declaran al final de esta ficha.

Requisito	Implementación	Evidencia
Stack justificado	FastAPI + SQLAlchemy async + PostgreSQL 16 + React 18/TS + n8n + LangChain/Ollama	§7 y requirements/package.json
PostgreSQL principal	13 tablas, 5 vistas, 11 enums, JSONB/INET/ARRAY, triggers	§2, db/schema.sql
Dashboard de métricas	8 endpoints; KPIs, tendencia, severidad, MTTT, SLA, FP y orígenes	§3.10, §4.7.2
Reportes PDF/Word/Excel	ReportLab, python-docx y openpyxl; 3 tipos × 3 formatos	§3.11, pruebas
Usuarios, roles y perfiles (RBAC)	5 roles, 26 permisos, matriz editable y validación por endpoint	§3.5
Bitácora de auditoría	ENTER/EXIT por módulo + eventos; usuario, fecha, hora, acción, IP y hash chain	§3.6
Arquitectura	Capas, componentes, interacciones, ADR, despliegue y riesgos	§1
Modelo de datos y semillas	Diccionario, ER, triggers, 728 alertas y 647 triajes	§2
Backend completo	7 routers, servicios, middleware, 14 pruebas de integración	§3
Frontend completo	9 páginas, rutas protegidas, cliente con refresh, build verificado	§4
Flujo de nodos	21 nodos, 10 operativos, JSON importable y SQL validado	§5 y n8n/
Datasets públicos	25 fuentes en 4 categorías, licencias y protocolo de evaluación	§6
Stack detallado	Versiones, alternativas, dimensionamiento y coste	§7

Verificación ejecutada

Componente	Verificación	Resultado
PostgreSQL	schema.sql + seed.sql en PostgreSQL 16	13 tablas · 728 alertas · 647 triajes
Backend	pytest (integración, BD real)	14 passed
API	login, RBAC, ingesta, dedup, triaje, feedback, incidente, reportes, auditoría	OK
Reportes	PDF, DOCX y XLSX generados y descargados	Archivos válidos
Auditoría	GET /audit/verify-chain	Cadena íntegra
Frontend	tsc --noEmit + vite build	0 errores TS · ~867 kB
UI	Chrome headless	9 capturas
n8n	Import en n8n:latest + SQL contra PostgreSQL	21 nodos · SQL válido

Limitaciones declaradas

- Ollama no estaba disponible en el entorno de verificación: el triaje respondió con el motor heurístico (status=degraded). El camino LLM está implementado y se activa al detectar Ollama.
- El workflow n8n no se ejecutó de punta a punta porque requiere crear la credencial PostgreSQL y las variables de Slack/osTicket. Sí se importó y se validaron sus dos consultas SQL.
- Las APIs de threat intelligence requieren claves; sin ellas se marcan como skipped y baja la confianza.
- No hay pruebas E2E de navegador automatizadas ni auditoría formal de accesibilidad; se documentan como siguiente paso.

1. Arquitectura del sistema

> Caso de uso 1 — Triaje automático de alertas SOC («SOC-Lite»), nivel de impacto **crítico**.
>
> Solución orientada a PYME, mediana empresa y organismo público, desplegable on-premise.

1.1 Resumen ejecutivo

SOC-Lite convierte el flujo tradicional SIEM → cola infinita → analista saturado en un pipeline SIEM → normalización → enriquecimiento → veredicto → acción con trazabilidad completa. La decisión de arquitectura central es **separar la orquestación (n8n)** de la **lógica de decisión (LangChain) y del** sistema de registro (PostgreSQL + FastAPI)**: n8n integra, LangChain razona, FastAPI valida y persiste, React presenta.
El sistema no intenta sustituir al analista. El agente **propone** un veredicto, una severidad y unas acciones; el analista confirma o corrige con un clic. Esa validación alimenta la memoria de falsos positivos y las métricas de calidad.

Objetivos medibles

Objetivo	Métrica	Línea base (sin SOC-Lite)	Meta
Reducir el tiempo hasta el triaje	MTTT promedio / P95	2–8 h	< 30 min promedio, P95 < 60 min
Filtrar ruido	Tasa de falsos positivos identificados	70–90 % sin priorizar	> 70 % del ruido clasificado como benigno/FP
Operación continua	Cobertura horaria	8×5	24×7 con escalado humano
Trazabilidad	Eventos auditados	Parcial	100 % de accesos y acciones críticas
Evidencia normativa	Controles cubiertos	—	NIST CSF DE.AE/RS.AN/RS.MA; ISO 27001 A.8.15/A.8.16/A.5.24–A.5.28

Alcance

Dentro del alcance: ingesta Wazuh/OPNsense/Suricata, deduplicación, enriquecimiento con VirusTotal/AbuseIPDB/GeoIP, triaje LLM local con fallback, dashboard, alertas, incidentes, usuarios/roles, auditoría, reportes PDF/Word/Excel, workflow n8n y despliegue Docker.
Fuera del alcance de esta versión: respuesta automática activa (bloqueo/aislamiento), integración EDR bidireccional, correlación multi-tenant, firma digital de reportes, SSO/SAML, aplicación móvil nativa. Se documentan como evolución.

1.2 Vista de contexto (C4 · nivel 1)

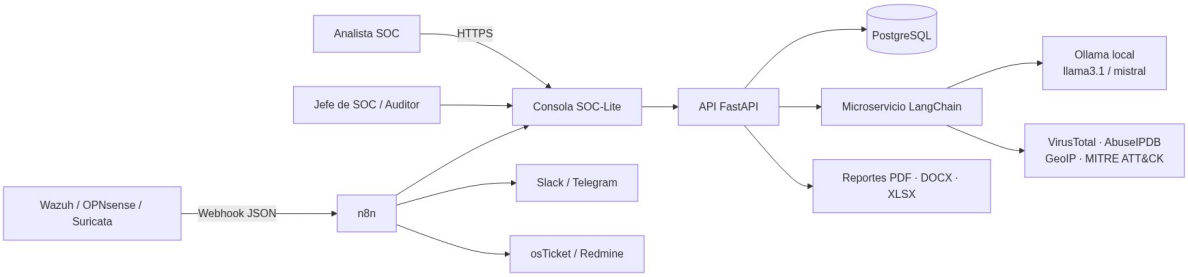


Figura 1. Diagrama de arquitectura / flujo

Fronteras de confianza

1. **Internet → SIEM:** tráfico no confiable; el SIEM ya aplica sus reglas.
2. **SIEM → n8n:** webhook autenticado por red interna y clave de ingesta en la API.
3. **n8n → LangChain:** red interna Docker; sin exposición pública.
4. **LangChain → proveedores externos:** sólo se envía el **indicador** (IP/dominio), nunca el payload completo ni nombres de activos.
5. **Usuario → consola:** JWT, RBAC y bitácora; TLS en el proxy inverso.

1.3 Vista de contenedores (C4 · nivel 2)

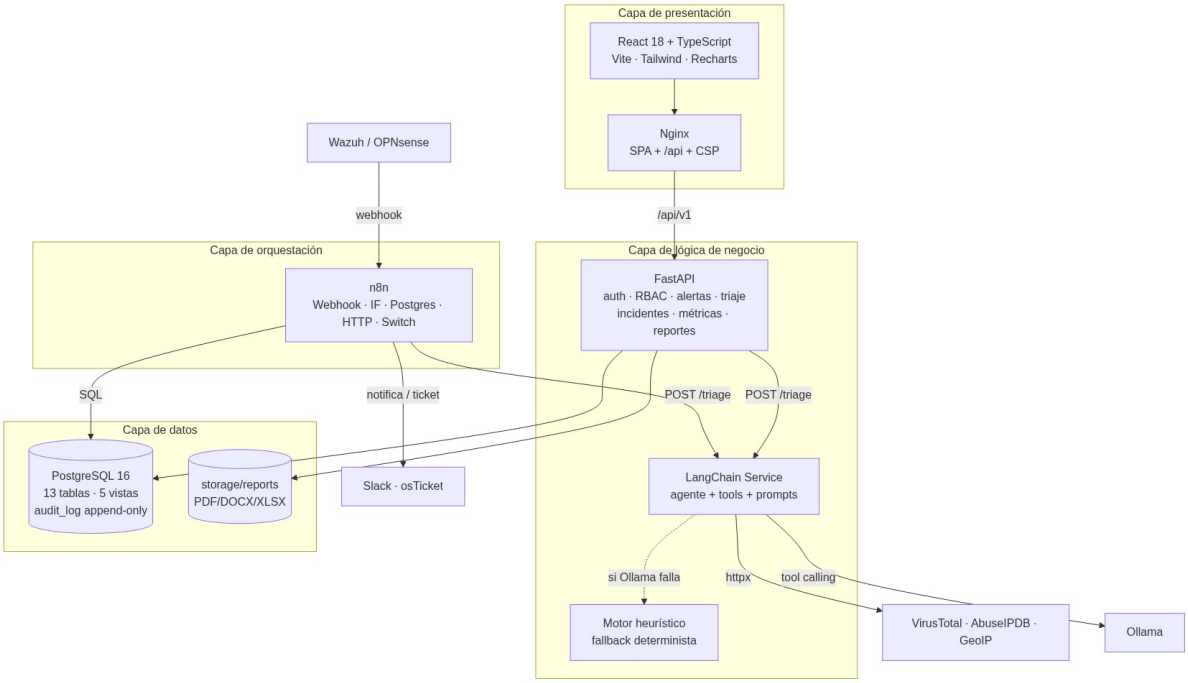


Figura 2. Diagrama de arquitectura / flujo

Responsabilidades por capa

Capa	Componente	Responsabilidad	No debe hacer
Presentación	React + Nginx	Visualizar, filtrar, capturar decisiones, descargar reportes	Reglas de negocio ni acceso directo a BD
Orquestación	n8n	Integrar, normalizar, deduplicar, enrutar, reintentar, notificar	Clasificar con LLM ni persistir lógica compleja
Lógica	FastAPI	Validar, autorizar, persistir, calcular métricas, generar reportes	Ejecutar acciones ofensivas o de contención

Capa	Componente	Responsabilidad	No debe hacer
Lógica IA	LangChain	Enriquecer, razonar, proponer veredicto y acciones	Escribir en BD, ejecutar comandos, decidir sin validación
Datos	PostgreSQL	Sistema de registro, integridad, auditoría inmutable	Lógica de presentación

---

1.4 Componentes

1.4.1 Capa de presentación

- **React 18 + TypeScript + Vite**: SPA con routing protegido por permisos.
- **Tailwind CSS**: sistema de diseño oscuro orientado a operación 24/7, con estados de severidad consistentes en toda la aplicación.
- **TanStack Query**: caché, reintentos y refresco de datos del servidor; separa el estado remoto del estado local.
- **Recharts**: tendencia diaria, distribución por severidad, top orígenes, MTTT/SLA y tasa de FP.
- **Axios con interceptor**: adjunta el JWT y renueva la sesión de forma transparente con rotación de refresh token; si la renovación falla, cierra sesión y redirige al login.
- **Nginx**: sirve la SPA, proxya `/api` al backend, aplica CSP y cabeceras de seguridad.

1.4.2 Capa de orquestación (n8n)

Función	Implementación
Entrada	Nodo Webhook `POST /webhook/soc-lite-ingest` con `responseMode=responseNode`
Normalización	Nodo Code: mapea Wazuh/OPNsense, calcula severidad y huella SHA-256
Filtro	Nodo IF: descarta `severity < medium` (ruido informativo)
Persistencia	Nodo PostgreSQL con `INSERT ... ON CONFLICT (fingerprint) DO UPDATE`
Deduplicación	Huella única en ventana de 15 min + contador `dedup_count`
Triaje	Nodo HTTP con reintentos (`maxTries=2`) al microservicio LangChain
Enrutamiento	Nodo Switch por severidad (alta/crítica, media, baja/info)
Persistencia del triaje	Nodo PostgreSQL con CTE `INSERT ... RETURNING` + `UPDATE alerts`
Notificación	HTTP a Slack sólo para alta/crítica
Ticketing	HTTP a osTicket/Redmine para alta/crítica
Auditoría	HTTP a `POST /api/v1/audit/events` con `X-Ingest-Key`
Resiliencia	Si LangChain falla, la alerta vuelve a `pending` y se audita el fallo

El detalle nodo a nodo está en 05-workflow-n8n.md; el JSON importable en `n8n/soc_lite_triage_workflow.json`.

1.4.3 Capa de lógica (FastAPI)

Módulo	Responsabilidad
`core`	Configuración 12-factor, JWT, bcrypt, RBAC, errores uniformes, logging JSON

Módulo	Responsabilidad
middleware	Request-ID, cabeceras de seguridad y bitácora ENTER/EXIT por módulo
models	ORM SQLAlchemy 2 tipado, enums nativos de PostgreSQL, relaciones
schemas	Contratos Pydantic v2 de entrada/salida y validación estricta
api/v1/routers	7 routers: auth, users, alerts, incidents, dashboard, reports, audit
services	Normalización, cliente de triaje, métricas SQL, reportes, auditoría
tests	14 pruebas de integración contra PostgreSQL real

Decisiones clave del backend

6. **Async de punta a punta** (asyncpg + SQLAlchemy async) para sostener I/O concurrente sin inflar workers.
7. **La API es la única puerta de escritura de negocio.** n8n puede escribir alertas/triajes por SQL por rendimiento, pero las acciones sensibles (feedback, usuarios, reportes) pasan por API.
8. **Fail-open controlado en triaje**: si el LLM no responde, se usa heurística local y se marca degraded; la alerta nunca queda sin clasificar.
9. **El agente no ejecuta**: no hay herramientas de bloqueo, aislamiento ni borrado. Las recomendaciones son texto estructurado que el analista decide.

1.4.4 Capa de datos

- **PostgreSQL 16** como sistema de registro.
- Tipos nativos: JSONB (payloads y acciones), INET (IPs), ARRAY (tags y MITRE), ENUM (estados), NUMERIC (confianza), TIMESTAMPTZ.
- **13 tablas**, **5 vistas**, **11 enums**, triggers de integridad y cadena de hashes.
- **`audit_log` append-only**: triggers que rechazan UPDATE y DELETE; cada fila enlaza el hash de la anterior.
- **Reportes**: archivos en storage/reports y metadatos en la tabla reports (autor, parámetros, tamaño, estado).

Ver 02-modelo-datos.md.

1.4.5 Capa de IA (LangChain)

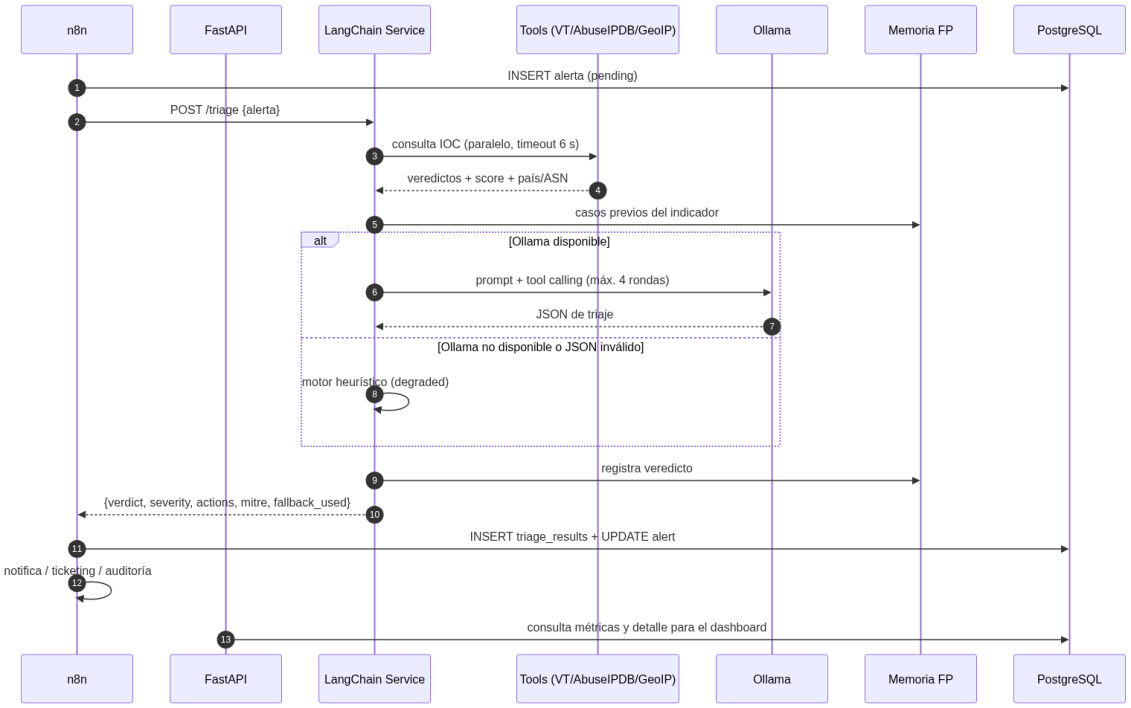


Figura 3. Diagrama de arquitectura / flujo

Controles del agente

Riesgo	Control
Inyección de prompts en logs	El payload se delimita como dato no confiable; el sistema instruye a ignorar órdenes embebidas
Alucinación de IOCs	Las herramientas devuelven `skipped`/`error`; el prompt prohíbe inventar evidencia
Salida no estructurada	Validación Pydantic + reintento de reparación + fallback heurístico
Acciones peligrosas	El agente sólo recomienda; no tiene herramientas de ejecución
Fuga de datos	Ollama local por defecto; a proveedores externos sólo se envía el indicador
Deriva del modelo	`prompt_version` y `model` se persisten en cada triaje

1.5 Flujo extremo a extremo

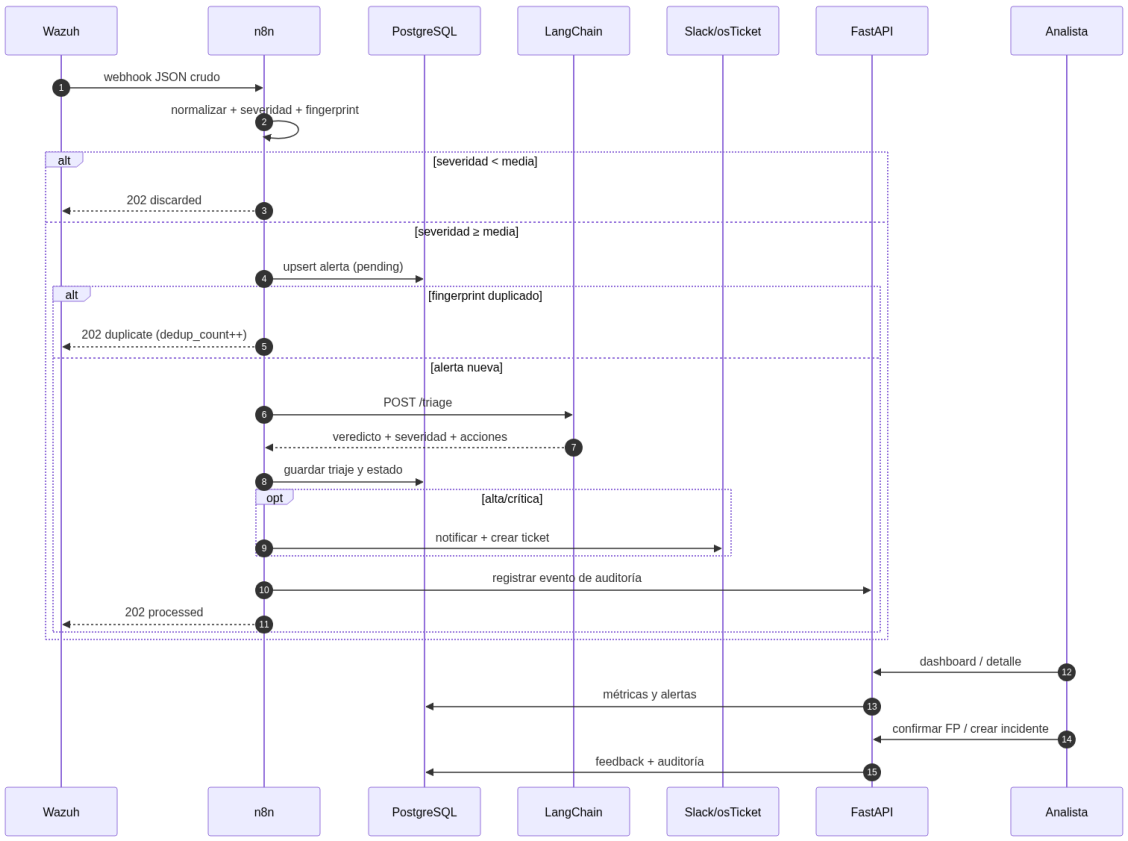


Figura 4. Diagrama de arquitectura / flujo

Presupuesto de latencia (objetivo P95 < 60 s):

Etapa	Tiempo típico	Peor caso
Webhook + normalización	< 100 ms	300 ms
Upsert PostgreSQL	< 50 ms	200 ms
Enriquecimiento (3 APIs, timeout 6 s)	0.5–3 s	6 s
LLM local (8B, 4 rondas)	5–25 s	90 s
Persistencia + notificación	< 500 ms	2 s
Total	**6–30 s**	**~100 s** (con fallback)

1.6 Decisiones de arquitectura (ADR)

ADR-01 · n8n para orquestación, no para la lógica de decisión

- **Contexto:** el equipo domina n8n; se necesita integrar Wazuh, HTTP, SQL, Slack y osTicket con reintentos y visibilidad operativa.
- **Decisión:** n8n orquesta el pipeline; la clasificación vive en un microservicio LangChain y la persistencia de negocio en FastAPI.
- **Consecuencias:** + velocidad de integración y mantenimiento por analistas; − una pieza más que desplegar; se mitiga con healthchecks y contrato HTTP versionado.
- **Alternativas descartadas:** todo en Python (menos visible para operación); todo en n8n (imposible mantener prompts, tools y tests con calidad).

ADR-02 · LangChain en microservicio FastAPI, no LangFlow

- **Contexto:** se requiere tool calling, memoria de casos, validación de salida y control fino de prompts; LangFlow es excelente para prototipos visuales pero incómodo para tests y versionado.
- **Decisión:** servicio FastAPI independiente con LangChain, prompts versionados y Pydantic.
- **Consecuencias:** + testeable, versionable y desplegable por separado; − requiere conocimiento de Python por parte del equipo.
- **Mitigación:** el contrato `/triage` es simple y el servicio se puede reemplazar sin tocar n8n.

ADR-03 · Ollama local por defecto, OpenAI opcional

- **Contexto:** las alertas contienen IPs internas, nombres de activos y usuarios; enviarlas a un proveedor cloud puede violar la política de datos.
- **Decisión:** `llama3.1:8b`/`mistral` en Ollama; el proveedor es configurable por variables.
- **Consecuencias:** + soberanía del dato y coste marginal cero; − requiere hardware (8–16 GB RAM o GPU) y la calidad del 8B es inferior a modelos frontier.
- **Mitigación:** fallback heurístico, revisión humana obligatoria en `inconclusive`/`suspicious` y posibilidad de cambiar a un modelo mayor en el mismo endpoint.

ADR-04 · PostgreSQL con JSONB + tipos nativos

- **Contexto:** el payload del SIEM es heterogéneo; las consultas del dashboard son agregaciones relacionales con percentiles.
- **Decisión:** columnas tipadas para lo consultable, `JSONB` para el crudo, `INET` para IPs.
- **Consecuencias:** + integridad y rendimiento; − migraciones más cuidadosas.
- **Alternativas descartadas:** MongoDB (peor para métricas y joins); sólo texto (pierde validación).

ADR-05 · RBAC en base de datos, no en el frontend

- **Contexto:** múltiples perfiles (admin, jefe de SOC, analista, auditor, consulta).
- **Decisión:** roles/permisos en tablas, validados por dependencia en cada endpoint; el frontend sólo oculta acciones.
- **Consecuencias:** + una sola fuente de verdad; − una consulta extra por request (mitigada con eager loading y caché de JWT).
- **Nota:** los permisos también viajan en el JWT para que la UI pinte el menú, pero **el backend nunca confía en ellos**: los recalcula desde la base.

ADR-06 · Bitácora append-only con cadena de hashes

- **Contexto:** ISO 27001 A.8.15 exige registros protegidos contra alteración.
- **Decisión:** triggers que bloquean `UPDATE`/`DELETE` y hash encadenado `prev_hash + campos`, con una secuencia (`chain_seq`) asignada dentro del lock para preservar el orden bajo concurrencia.
- **Consecuencias:** + evidencia verificable; − crece indefinidamente (se mitiga con particionado y archivado, ver §1.9).
- **Limitación:** la cadena detecta alteraciones, no las impide si un atacante tiene privilegios de superusuario; se recomienda exportar periódicamente a WORM/S3 Object Lock.

ADR-07 · Fail-open en triaje, fail-closed en autorización

- **Contexto:** disponibilidad 24/7 vs. seguridad.
- **Decisión:** si el LLM falla, se sigue triando con heurística; si falla la autenticación o el permiso, se deniega.

- **Consecuencias:** + ninguna alerta se pierde; − un triaje degradado puede requerir revisión humana (se marca needs_human).

ADR-08 · Reportes generados en servidor

- **Contexto:** PDF/Word/Excel deben ser reproducibles y auditables, no una captura del navegador.
- **Decisión:** ReportLab, python-docx y openpyxl en el backend; archivo persistido + fila en reports.
- **Consecuencias:** + consistencia y trazabilidad; − generación síncrona de 1–3 s (aceptable; se puede mover a cola si crece).

1.7 Despliegue

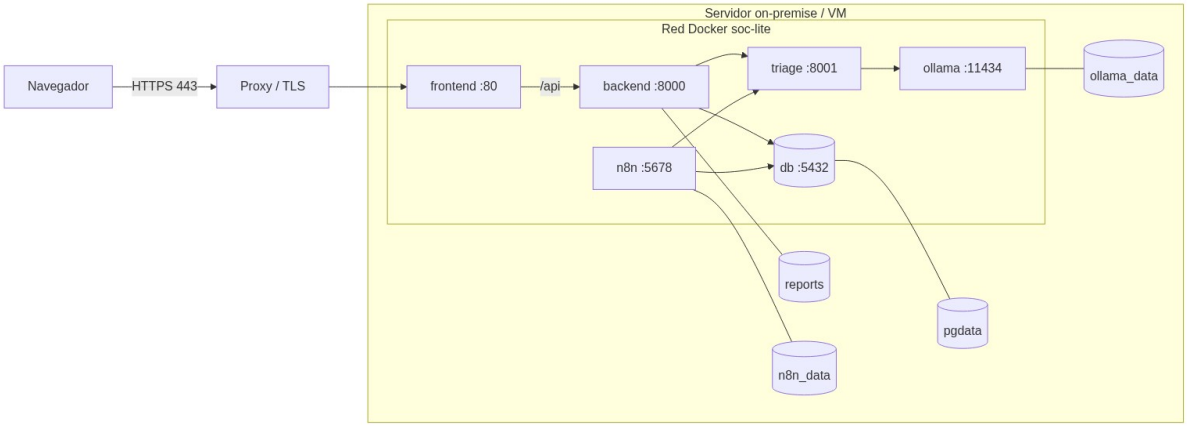


Figura 5. Diagrama de arquitectura / flujo

Servicio	Imagen	Puerto host	Persistencia
db	postgres:16-alpine	5439 → 5432	volumen pgdata
backend	build ./backend	8010	volumen reports
triage	build ./langchain-service	8011	data/ (memoria de casos)
frontend	build ./frontend (Nginx)	8090	—
n8n	n8nio/n8n:latest	5679	volumen n8n_data
ollama	ollama/ollama:latest (perfil llm)	11434	volumen ollama_data

Requisitos mínimos: 4 vCPU, 8 GB RAM, 80 GB SSD (sin Ollama); 8 vCPU, 16 GB RAM y GPU opcional con Ollama. Para 5 000 alertas/día el cuello de botella es el LLM, no PostgreSQL.

1.8 Seguridad y cumplimiento

Controles técnicos

Control	Implementación
Autenticación	JWT HS256, access 30 min, refresh 7 días con rotación y detección de reutilización
Contraseñas	bcrypt coste 12; política de 10+ caracteres con mayúscula, dígito y símbolo
Autorización	RBAC de 26 permisos, validado por endpoint; admin inmutable

Control	Implementación
Auditoría	ENTER/EXIT por módulo + eventos de negocio; hash chain; export CSV/XLSX
Integridad	Triggers append-only; restricciones CHECK, FKs, enums
Ingesta	X-Ingest-Key con secrets.compare_digest; límite de tamaño en Nginx
Cabeceras	CSP, X-Frame-Options: DENY, nosniff, Referrer-Policy, HSTS en producción
Secretos	Variables de entorno; .env fuera de Git; claves de API nunca en la BD
Datos personales	Minimización: el LLM recibe sólo indicadores, no el payload completo
Prompt injection	Delimitación de datos + instrucciones + validación de salida

Mapeo normativo

NIST CSF 2.0	ISO/IEC 27001:2022	Cómo lo cubre SOC-Lite
DE.AE-02 (analizar eventos)	A.8.16 (monitoreo)	Enriquecimiento + veredicto + severidad
DE.AE-04 (impacto estimado)	A.5.25 (evaluación de eventos)	Criticidad del activo + confianza + acciones
DE.AE-06 (alertas)	A.8.15 (registro de eventos)	Ingesta, dedup, notificación y bitácora
RS.AN-03 (investigación)	A.5.26 (respuesta a incidentes)	Detalle, evidencia, feedback, ticket
RS.MA-01 (gestión de incidentes)	A.5.24 (plan de respuesta)	Incidentes con prioridad, SLA y responsable
GV.OC-03 (requisitos legales)	A.5.28 (recolección de evidencia)	Exportación auditada y cadena de hashes
PR.AA-05 (gestión de accesos)	A.8.2/A.8.3 (privilegios)	RBAC + usuarios + revisiones auditadas

> Nota: el mapeo es de **apoyo**; la certificación requiere auditoría formal, políticas y evidencia organizacional que este software no sustituye.

1.9 Operación, riesgos y evolución

Riesgos y mitigaciones

Riesgo	Prob.	Impacto	Mitigación
LLM alucina un veredicto	Media	Alto	Herramientas con evidencia, validación Pydantic, revisión humana, confianza explícita
Fuga de datos a API externa	Baja	Crítico	Ollama local; sólo se envía el indicador; claves opcionales
Inyección de prompt en un log	Media	Alto	Datos delimitados, instrucciones anti-inyección, salida validada, sin herramientas de ejecución
Saturación de la cola por picos	Media	Medio	Dedup, filtro de severidad, pool async, índices, escalado horizontal de API
Pérdida de la BD	Baja	Crítico	Backups diarios + WAL; réplica de lectura; pruebas de restauración
Crecimiento de audit_log	Alta	Medio	Particionado mensual, archivado a object storage, retención configurable
Dependencia de n8n	Media	Medio	Workflow versionado en Git; endpoint de ingesta alternativo en FastAPI

Riesgo	Prob.	Impacto	Mitigación
Modelo desactualizado	Media	Medio	model + prompt_version persistidos; evaluación con golden set

Retención recomendada

Dato	Retención	Justificación
alerts / triage_results	12 meses online, 3 años archivados	Investigación y tendencias
audit_log	3–5 años (según regulación)	Evidencia legal/ISO
enrichment_data	90 días (TTL del proveedor)	Evita sanciones por cachear datos con licencia
Reportes	2 años	Gestión y auditoría
Memoria de triaje	500 entradas / rotación	Privacidad y rendimiento

Evolución

10. **Corto plazo:** rate limiting en ingesta, MFA TOTP, SSO/OIDC, notificaciones Teams.
11. **Mediano plazo:** casos de uso 2–5 (vulnerabilidades, phishing, cumplimiento, respuesta), reutilizando RBAC, auditoría, dashboard y reportes.
12. **Largo plazo:** correlación multi-tenant, evaluación automática del modelo con datasets etiquetados y ajuste de prompts por retroalimentación.

2. Modelo de datos PostgreSQL y datos semilla

> Motor: **PostgreSQL 16**. Esquema completo en `db/schema.sql`; datos de demostración en `db/seed.sql`.

2.1 Principios de diseño

13. **Lo consultable es relacional; lo heterogéneo es `JSONB`.** IPs en INET, fechas en TIMESTAMPTZ, estados en ENUM, técnicas MITRE en ARRAY.
14. **El histórico no se sobrescribe.** Cada ejecución de triaje crea una fila nueva; la alerta conserva la última como vista materializada lógica (LATERAL … ORDER BY triaged_at DESC).
15. **La auditoría es append-only.** UPDATE/DELETE bloqueados por trigger y hash encadenado.
16. **Integridad en la base, no sólo en la aplicación.** CHECK, UNIQUE, FK con ON DELETE explícito y enums nativos.
17. **Consultas del dashboard en SQL.** percentile_cont, FILTER, generate_series y CTEs; PostgreSQL calcula, Python sólo serializa.

2.2 Diagrama entidad-relación

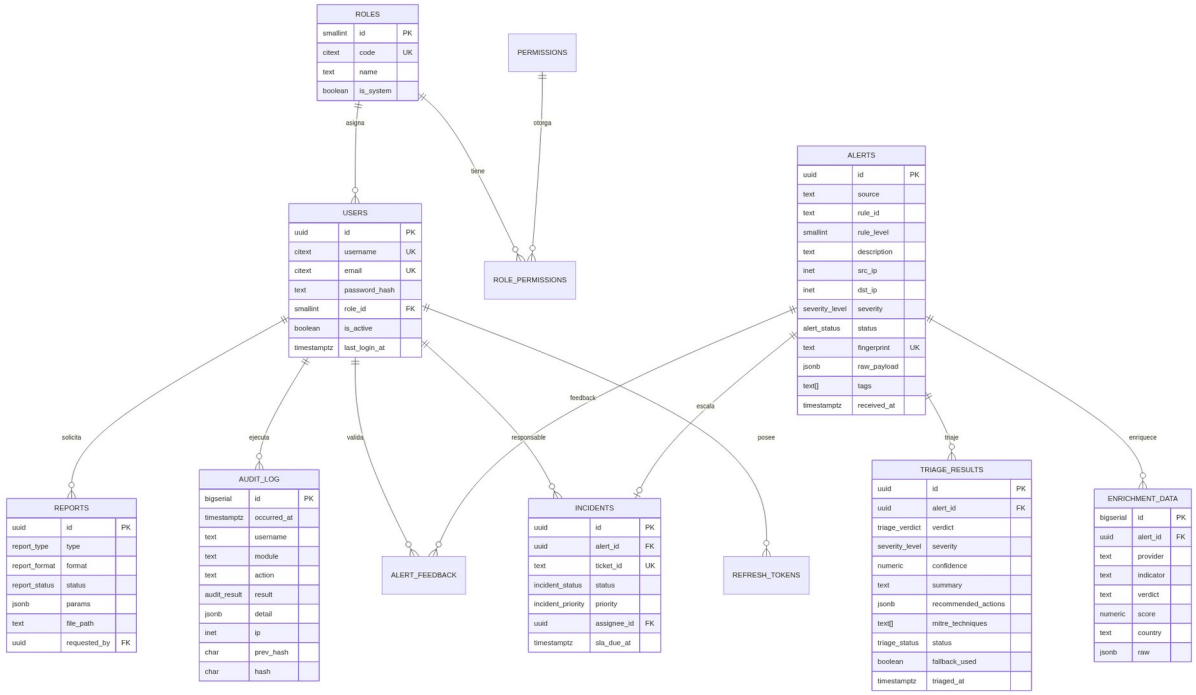


Figura 6. Diagrama de arquitectura / flujo

2.3 Tipos enumerados

Enum	Valores	Uso
severity_level	critical, high, medium, low, info	Alertas y triaje
alert_status	pending, triaging, triaged, in_review, escalated, closed, false_positive, discarded	Ciclo de vida
triage_verdict	true_positive, false_positive, suspicious, benign, inconclusive	Decisión del agente
triage_status	pending, completed, degraded, failed	Calidad de la ejecución
incident_status	open, investigating, contained, resolved, closed	Gestión del incidente
incident_priority	P1, P2, P3, P4	Prioridad y SLA
audit_result	success, failure, denied	Resultado auditado
report_type	operational, management, audit	Tipo de reporte
report_format	pdf, docx, xlsx	Formato de salida
report_status	queued, generating, ready, failed	Estado de generación
integration_type	siem, edr, ticketing, chat, threat_intel, llm	Catálogo de integraciones

¿Por qué enums nativos? Validación en la base, índices compactos y consultas legibles (WHERE severity = 'critical'). El coste es una migración controlada al añadir valores (ALTER TYPE ... ADD VALUE); se documenta en §2.9.

2.4 Diccionario de tablas

2.4.1 roles

Columna	Tipo	Restricciones	Descripción
id	smallserial	PK	Identificador
code	citext	UNIQUE, NOT NULL	admin, soc_manager, analyst, auditor, viewer
name	text	NOT NULL	Nombre visible
description	text	—	Propósito del rol
is_system	boolean	NOT NULL, default false	Los roles de sistema no se eliminan
created_at	timestamptz	default now()	Alta

2.4.2 permissions y role_permissions

Columna	Tipo	Restricciones	Descripción
id	serial	PK	Identificador
code	text	UNIQUE, NOT NULL	alerts:read, reports:export…
module	text	NOT NULL	dashboard, alerts, triage, incidents, users, roles, audit, reports
action	text	NOT NULL	read, create, update, delete, export, run, feedback, close
description	text	—	Texto para la matriz de la UI

role_permissions(role_id, permission_id) es la tabla puente con PK compuesta y ON DELETE CASCADE.

2.4.3 users

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, gen_random_uuid()	Identificador
username	citext	UNIQUE, 3–64	Login insensible a mayúsculas
email	citext	UNIQUE, CHECK regex	Contacto
full_name	text	NOT NULL	Nombre visible
password_hash	text	NOT NULL	bcrypt (coste 12)
role_id	smallint	FK roles	Rol único por usuario
is_active	boolean	default true	Acceso habilitado
mfa_enabled	boolean	default false	Reservado para TOTP
last_login_at	timestamptz	—	Último acceso
created_at / updated_at	timestamptz	now() + trigger	Trazabilidad
deleted_at	timestamptz	—	Baja lógica

2.4.4 refresh_tokens

Columna	Tipo	Restricciones	Descripción
id	uuid	PK	Identificador

Columna	Tipo	Restricciones	Descripción
user_id	uuid	FK users ON DELETE CASCADE	Titular
token_hash	char(64)	UNIQUE	SHA-256 del token; nunca se guarda en claro
user_agent / ip	text / inet	—	Contexto de la sesión
expires_at	timestamptz	NOT NULL	Caducidad
revoked_at	timestamptz	—	Rotación / logout
created_at	timestamptz	now()	Emisión

2.4.5 alerts (tabla central)

Columna	Tipo	Restricciones	Descripción
id	uuid	PK	Identificador interno
external_id	text	—	ID en Wazuh/OPNsense
source	text	NOT NULL	wazuh, opnsense, suricata…
rule_id	text	—	Regla del SIEM
rule_level	smallint	CHECK 0–16	Nivel original
description	text	NOT NULL	Texto de la alerta
src_ip / dst_ip	inet	—	Origen/destino
src_port / dst_port	integer	CHECK 0–65535	Puertos
protocol	text	—	tcp/udp/icmp
agent_name	text	—	Activo afectado
asset_criticality	smallint	CHECK 1–5	Criticidad del activo
severity	severity_level	NOT NULL	Severidad efectiva
status	alert_status	NOT NULL, default pending	Ciclo de vida
fingerprint	text	UNIQUE, NOT NULL	SHA-256 de correlación (ventana 15 min)
raw_payload	jsonb	default {}	Payload original
tags	text[]	default {}	Etiquetas derivadas
dedup_count	integer	default 1	Repeticiones agregadas
occurred_at	timestamptz	—	Cuándo ocurrió en el SIEM
received_at	timestamptz	default now()	Cuándo llegó a SOC-Lite
last_seen_at	timestamptz	default now()	Última repetición
created_at	timestamptz	default now()	Alta

Índices: severity, status, received_at DESC, source, src_ip, agent_name, rule_id, GIN en tags, GIN trigram en description, UNIQUE en fingerprint.

Cálculo de la huella (services/alert_ingest.py):

```
fingerprint = sha256(source | rule_id | src_ip | dst_ip | external_id | floor(occurred_at / 15min))
```

La ventana de 15 minutos agrupa ráfagas del mismo evento sin impedir que el mismo ataque se registre al día siguiente.

2.4.6 enrichment_data

Columna	Tipo	Descripción
id	bigserial PK	—
alert_id	uuid FK CASCADE	Alerta enriquecida
provider	text	virustotal, abuseipdb, geoip, mitre, internal
indicator	text	IP, dominio, hash, URL
indicator_type	text	ip, domain, hash, url, host, technique
verdict	text	malicious, suspicious, clean, unknown
score	numeric(5,2)	0–100
country / asn	text	Geolocalización
raw	jsonb	Respuesta original saneada
fetched_at / expires_at	timestamptz	TTL del proveedor

2.4.7 triage_results

Columna	Tipo	Descripción
id	uuid PK	—
alert_id	uuid FK CASCADE	Alerta
verdict	triage_verdict	Decisión
severity	severity_level	Severidad sugerida
confidence	numeric(4,3)	0.000–1.000
summary	text	Resumen operativo en español
reasoning	text	Justificación
recommended_actions	jsonb	[{action, priority, owner}]
mitre_techniques	text[]	T1110, T1486…
model	text	llama3.1:8b o heuristic-engine
prompt_version	text	v1.2.0
latency_ms	integer	Duración
status	triage_status	completed / degraded / failed
needs_human	boolean	Requiere revisión
fallback_used	boolean	Se usó heurística
raw_response	jsonb	Respuesta del modelo (saneada)
triaged_at	timestamptz	Momento

Índices: (alert_id, triaged_at DESC), verdict, triaged_at DESC.

2.4.8 alert_feedback

Columna	Tipo	Descripción
id	bigserial PK	—
alert_id	uuid FK CASCADE	Alerta validada
user_id	uuid FK SET NULL	Analista
is_false_positive	boolean	Decisión humana

Columna	Tipo	Descripción
comment	text	Justificación
created_at	timestamptz	Fecha/hora

UNIQUE(alert_id, user_id): una validación por analista y alerta (se actualiza si cambia).

2.4.9 incidents

Columna	Tipo	Descripción
id	uuid PK	—
alert_id	uuid FK SET NULL	Alerta origen
ticket_id	text UNIQUE	OSK-000123 u osTicket externo
title / description	text	Contenido
status	incident_status	Ciclo de vida
priority	incident_priority	P1–P4
assignee_id	uuid FK SET NULL	Responsable
sla_due_at	timestamptz	Vencimiento (4/8/24/72 h según prioridad)
opened_at / closed_at	timestamptz	Fechas
resolution	text	Cierre
created_at / updated_at	timestamptz	Trazabilidad

2.4.10 audit_log (append-only)

Columna	Tipo	Descripción
id	bigserial PK	Identificador interno
chain_seq	bigint UNIQUE	Posición en la cadena; la asigna el trigger bajo advisory lock
occurred_at	timestamptz	Fecha y hora del evento
user_id	uuid FK SET NULL	Usuario (si existe)
username	text	Copia desnormalizada para conservar evidencia
module	text	Módulo (alerts, triage, auth...)
action	text	ENTER, EXIT, LOGIN, ALERT_INGEST, TRIAGE_RUN...
entity_type / entity_id	text	Entidad afectada
result	audit_result	success, failure, denied
detail	jsonb	Contexto (cambios, parámetros, motivo)
ip	inet	Origen
user_agent	text	Cliente
request_id	text	Correlación con logs
duration_ms	integer	Duración (EXIT)
prev_hash	char(64)	Hash anterior
hash	char(64)	Hash de esta fila

Trigger `fn_audit_chain`: bajo un advisory lock asigna chain_seq con la secuencia

audit_log_chain_seq_seq (no usa id, porque bigserial se asigna antes del trigger y dos transacciones concurrentes pueden intercalar sus ids), toma el hash previo y concatena prev_hash|occurred_at|user_id|username|module|action|entity_id|result|detail y calcula SHA-256 con pgcrypto.

Trigger `fn_audit_immutable`: lanza excepción ante UPDATE o DELETE.

Limitación honesta: un superusuario puede deshabilitar triggers; para evidencia legal se recomienda exportar periódicamente a almacenamiento WORM.

2.4.11 integrations

Catálogo de conectores: name, type (siem, edr, ticketing, chat, threat_intel, llm), config JSONB (referencias a variables de entorno, **nunca secretos en claro**), enabled, last_check_at.

2.4.12 reports

Columna	Tipo	Descripción
id	uuid PK	—
type	report_type	operacional / gestión / auditoría
format	report_format	pdf / docx / xlsx
status	report_status	queued / generating / ready / failed
params	jsonb	Periodo, título, solicitante
file_path / file_size	text / bigint	Archivo generado
requested_by	uuid FK SET NULL	Autor
created_at / completed_at	timestamptz	Fechas
error	text	Detalle si falló

2.5 Vistas

Vista	Propósito
v_alert_overview	Alerta + último triaje + incidente + MTTT en minutos
v_incident_overview	Incidente + severidad/descripción de la alerta + nombre del responsable
v_dashboard_daily	Serie diaria de los últimos 30 días por severidad
v_mttt_stats	Promedio, P95, mínimo y máximo del MTTT por severidad
v_audit_recent	Últimos 500 eventos con hash corto (para UI)

Ejemplo de v_alert_overview:

```
SELECT a.id, a.severity, t.verdict, t.confidence,
    EXTRACT(EPOCH FROM (t.triaged_at - a.received_at)) / 60.0 AS mttt_minutes
FROM alerts a
LEFT JOIN LATERAL (
  SELECT * FROM triage_results tr
  WHERE tr.alert_id = a.id
  ORDER BY tr.triaged_at DESC LIMIT 1
) t ON true;
```

2.6 Triggers y funciones

Objeto	Tipo	Función
trg_audit_chain	BEFORE INSERT en audit_log	Calcula prev_hash y hash con advisory lock
trg_audit_no_update	BEFORE UPDATE/DELETE	Bloquea modificaciones
trg_users_touch / trg_incidents_touch	BEFORE UPDATE	Actualiza updated_at
trg_alert_dedup	BEFORE INSERT	Ajusta last_seen_at si es duplicado agregado
generate_synthetic_alerts(count, days)	Función PL/pgSQL	Genera alertas, triajes, feedback e incidentes realistas

---

2.7 Datos semilla

Roles y permisos

Rol	Permisos	Usuarios demo
admin	26 (todos)	admin
soc_manager	22 (operación + lectura de usuarios/roles)	gerente
analyst	13 (alertas, triaje, incidentes, reportes)	analista1, analista2
auditor	10 (lectura + auditoría + reportes)	auditor
viewer	4 (solo lectura)	visor

Volumen generado

Tabla	Filas	Detalle
roles	5	Roles de sistema
permissions	26	8 módulos × acciones
users	6	Contraseñas bcrypt vía pgcrypto
alerts	728	8 curadas + 720 sintéticas en 60 días
triage_results	647	~89 % de las alertas triadas
incidents	33	Solo alta/crítica
enrichment_data	9	IOC de las alertas curadas
alert_feedback	~200	Validaciones humanas

La distribución sintética es deliberadamente realista: **> 40 % benigno**, ~12 % falso positivo, ~20 % verdadero positivo y el resto sospechoso/inconcluso. Así el dashboard muestra el problema que SOC-Lite resuelve (ruido) y no un mundo ideal.

Alertas curadas (historias de demostración)

ID corto	Escenario	Severidad	Veredicto	Incidente
...1101	Fuerza bruta SSH contra SRV-WEB01	Crítica	Verdadero positivo	OSK-10001
...1102	Ransomware en SRV-FILE01	Crítica	Verdadero positivo	OSK-10002
...1103	Escaneo de puertos bloqueado	Baja	Falso positivo	—

ID corto	Escenario	Severidad	Veredicto	Incidente
…1104	PowerShell con C2 en WKS-GERENCIA	Alta	Sospechoso	OSK-10003
…1105	Login fallido aislado interno	Media	Falso positivo	—
…1106	Explotación CVE-2024-4577	Alta	Verdadero positivo	OSK-10004
…1107	DNS a dominio recién registrado	Media	Benigno	—
…1108	Conexión bloqueada por firewall	Info	Benigno	—

Generador sintético

```
-- 5 000 alertas en 90 días para pruebas de carga
SELECT generate_synthetic_alerts(5000, 90);

-- Verificación
SELECT severity, count(*) FROM alerts GROUP BY 1 ORDER BY 2 DESC;
SELECT * FROM v_mttt_stats;
```

El generador usa setseed(0.4242) para reproducibilidad y respeta las FKs (usuarios, roles, incidentes, feedback).

2.8 Consultas de referencia

Último triaje por alerta

```
SELECT DISTINCT ON (tr.alert_id) tr.*
FROM triage_results tr
ORDER BY tr.alert_id, tr.triaged_at DESC;
```

MTTT promedio y P95

```
SELECT a.severity,
    ROUND(AVG(EXTRACT(EPOCH FROM (t.triaged_at - a.received_at)) / 60.0)::numeric, 1) AS avg_min,
    ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (
        ORDER BY EXTRACT(EPOCH FROM (t.triaged_at - a.received_at)) / 60.0)::numeric, 1) AS p95_min
FROM alerts a
JOIN LATERAL (SELECT * FROM triage_results tr WHERE tr.alert_id = a.id
          ORDER BY tr.triaged_at DESC LIMIT 1) t ON true
GROUP BY a.severity;
```

Cumplimiento de SLA por severidad

```
WITH latest AS (
 SELECT DISTINCT ON (alert_id) alert_id, triaged_at
 FROM triage_results ORDER BY alert_id, triaged_at DESC
)
SELECT a.severity,
    COUNT(*) AS total,
    COUNT(*) FILTER (
      WHERE EXTRACT(EPOCH FROM (l.triaged_at - a.received_at)) / 60.0
          <= CASE a.severity WHEN 'critical' THEN 15 WHEN 'high' THEN 30
                     WHEN 'medium' THEN 240 ELSE 1440 END
    ) AS dentro_sla
FROM alerts a JOIN latest l ON l.alert_id = a.id
GROUP BY a.severity;
```

Tasa de falsos positivos

```
SELECT ROUND(
  100.0 * COUNT(*) FILTER (WHERE t.verdict = 'false_positive') / NULLIF(COUNT(*), 0), 1
) AS fp_rate
FROM triage_results t
JOIN alerts a ON a.id = t.alert_id
WHERE a.received_at >= now() - interval '30 days';
```

Verificación de la cadena de auditoría

```
SELECT id, prev_hash, hash,
      encode(digest(
        COALESCE(prev_hash,'') || '|' || CAST(occurred_at AS text) || '|' ||
        COALESCE(CAST(user_id AS text),'') || '|' || COALESCE(username,'') || '|' ||
        module || '|' || action || '|' || COALESCE(entity_id,'') || '|' ||
        CAST(result AS text) || '|' || COALESCE(CAST(detail AS text),'{}'),
        'sha256'), 'hex') AS esperado
FROM audit_log ORDER BY id;
```

Si `hash <> esperado` o `prev_hash` no coincide con el hash anterior, la cadena está rota.

2.9 Operación del esquema

Aplicar desde cero

```
psql -h 127.0.0.1 -p 5439 -U soclite -d soclite -f db/schema.sql
psql -h 127.0.0.1 -p 5439 -U soclite -d soclite -f db/seed.sql
```

`schema.sql` es **destructivo en desarrollo**: elimina vistas, tablas y tipos antes de recrearlos.

En producción se debe usar Alembic (recomendado) o migraciones versionadas.

Añadir un valor a un enum

```
ALTER TYPE alert_status ADD VALUE IF NOT EXISTS 'suppressed';
-- No se puede usar el nuevo valor en la misma transacción; commit y luego usarlo.
```

Particionado de `audit_log` (crecimiento)

```
-- Ejemplo conceptual: partición mensual por occurred_at
CREATE TABLE audit_log_2026_10 PARTITION OF audit_log
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

Para adoptarlo, `audit_log` debe declararse `PARTITION BY RANGE (occurred_at)` y la PK incluir `occurred_at`. Se documenta como evolución, no está aplicado en esta versión.

Respaldos

```
pg_dump -Fc -h 127.0.0.1 -p 5439 -U soclite soclite > soclite_$(date +%F).dump
pg_restore -h 127.0.0.1 -p 5439 -U soclite -d soclite --clean --if-exists soclite_2026-10-20.dump
```

Frecuencia recomendada: diaria completa + WAL continuo; retención 30 días; prueba de restauración mensual.

2.10 Decisiones y límites

Decisión	Motivo	Límite
Un rol por usuario	Simplicidad operativa para PYME	Un usuario con necesidades mixtas requiere rol a medida
`JSONB` para el payload	Heterogeneidad de SIEM	Consultas analíticas sobre el crudo son más costosas
Enums nativos	Integridad y rendimiento	Añadir valores requiere migración

Decisión	Motivo	Límite
Baja lógica de usuarios	Preservar auditoría e historial	Hay que filtrar `deleted_at IS NULL` en todas las consultas
Hash chain en trigger	Integridad incluso fuera de la app	Serializa inserts de auditoría (advisory lock)
Sin particionado inicial	Simplicidad de despliegue	A gran volumen (> 10 M eventos) hay que particionar

3. Desarrollo del backend

> **Stack:** Python 3.11 · FastAPI 0.115 · SQLAlchemy 2.0 async · asyncpg · Pydantic v2 · PostgreSQL 16 · ReportLab · python-docx · openpyxl.
>
> Código en `backend/`; pruebas en `backend/tests/`.

3.1 Estructura

```
backend/
├── app/
│   ├── main.py                # create_app, middlewares, routers, lifespan
│   ├── core/
│   │   ├── config.py          # Settings (pydantic-settings, 12-factor)
│   │   ├── security.py        # bcrypt, JWT, refresh, política de contraseñas
│   │   ├── deps.py            # get_current_user, require_permission, paginación, IP
│   │   ├── permissions.py     # catálogo P() de 26 permisos
│   │   ├── errors.py          # AppError y handlers uniformes
│   │   └── logging.py         # JSON logs + request_id/username contextvars
│   ├── db/
│   │   ├── base.py            # DeclarativeBase + naming convention
│   │   └── session.py         # engine async, SessionLocal, get_db
│   ├── models/                # security, alert, incident, audit + enums
│   ├── schemas/               # common, auth, user, alert, dashboard, report
│   ├── api/v1/
│   │   ├── router.py          # agregador /api/v1
│   │   └── routers/           # auth, users, alerts, incidents, dashboard,
│   │                          # reports, audit, health
│   ├── services/
│   │   ├── alert_ingest.py    # normalización Wazuh/OPNsense + fingerprint
│   │   ├── triage_client.py   # cliente LangChain + fallback heurístico
│   │   ├── dashboard.py       # agregaciones SQL (MTTT, SLA, FP, top)
│   │   ├── reports.py         # PDF, DOCX, XLSX
│   │   └── audit.py           # record(), verify_chain()
│   └── middleware/
│       └── audit_middleware.py  # request-id, cabeceras, ENTER/EXIT
├── tests/                     # 14 pruebas de integración
├── requirements.txt
├── pytest.ini
└── Dockerfile
```

Patrón: routers delgados (validación + autorización + respuesta) y **servicios** con la lógica reutilizable. Los modelos nunca se serializan directamente: se usan esquemas Pydantic.

3.2 Configuración

`app/core/config.py` usa `pydantic-settings` con `.env`. Las variables principales:

Variable	Default	Descripción
`SECRET_KEY`	(dev)	Firma JWT; **obligatoria en producción**

Variable	Default	Descripción
ACCESS_TOKEN_EXPIRE_MINUTES	30	Vida del access token
REFRESH_TOKEN_EXPIRE_DAYS	7	Vida del refresh token
PASSWORD_MIN_LENGTH	10	Política de contraseña
DATABASE_URL	postgresql+asyncpg://...:5439/soclite	Conexión
INGEST_API_KEY	dev-ingest-key-change-me	Clave del webhook
TRIAGE_SERVICE_URL	http://127.0.0.1:8011	Microservicio LangChain
TRIAGE_TIMEOUT_SECONDS	25	Timeout del triaje
TRIAGE_FAIL_OPEN	true	Usar heurística si el LLM falla
DEDUP_WINDOW_MINUTES	15	Ventana de correlación
SLA_CRITICAL/HIGH/MEDIUM/LOW	15/30/240/1440	Objetivos en minutos
CORS_ORIGINS	localhost:8090	Orígenes permitidos
REPORTS_DIR	storage/reports	Salida de reportes

En lifespan se rechaza el arranque en producción si SECRET_KEY conserva el valor de desarrollo. /health devuelve estado de BD; /health/ready añade el estado del microservicio de triaje.

3.3 Ciclo de vida de un request

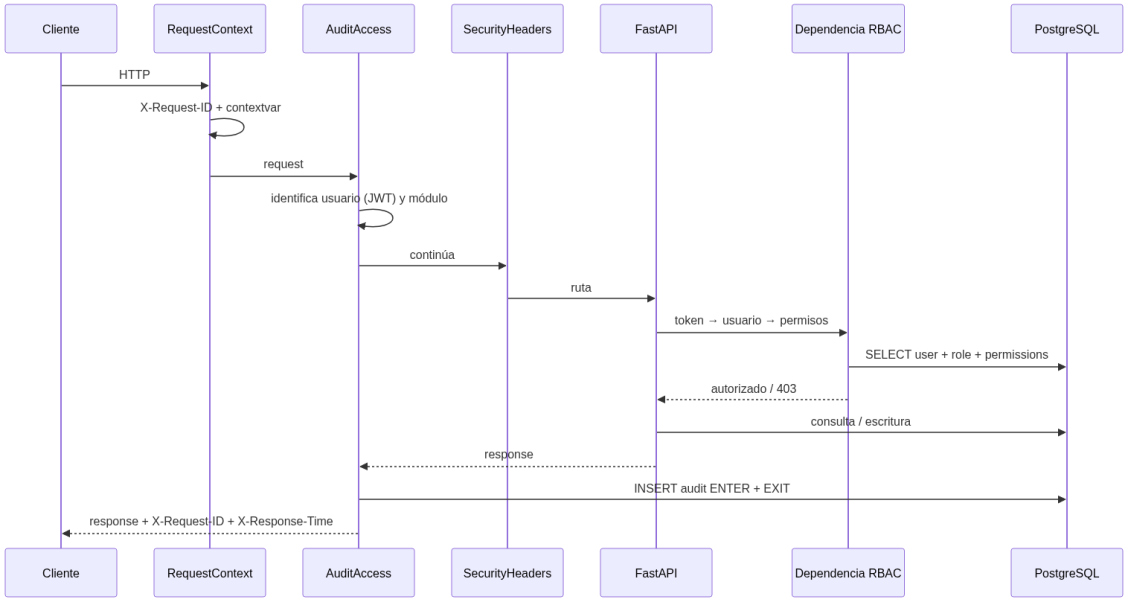


Figura 7. Diagrama de arquitectura / flujo

Orden real de middlewares (Starlette ejecuta el último añadido primero): RequestContext → AuditAccess → SecurityHeaders → CORS → router.

Detalle importante: la bitácora ENTER/EXIT se escribe **después** de la respuesta, con occurred_at del inicio y duration_ms medido. Así no se penaliza la latencia del endpoint, pero se conserva el instante de entrada. Si la escritura de auditoría falla, en desarrollo se registra el error sin romper la respuesta; en producción se propaga (fail-closed de evidencia).

3.4 Autenticación y sesión

Login

```
POST /api/v1/auth/login
Content-Type: application/json

{"username": "analista1", "password": "Analista123!"}
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer",
  "expires_in": 1800,
  "user": {
    "id": "...",
    "username": "analista1",
    "role": {"id": 3, "code": "analyst", "name": "Analista SOC"},
    "permissions": ["alerts:read", "triage:run", "..."]
  }
}
```

Claims del access token:

```
{
  "sub": "uuid-del-usuario",
  "type": "access",
  "username": "analista1",
  "role": "analyst",
  "permissions": ["alerts:read", "..."],
  "iat": 1792491284,
  "nbf": 1792491284,
  "exp": 1792493084,
  "jti": "...",
  "iss": "soc-lite"
}
```

Refresh con rotación

18. El cliente envía `POST /auth/refresh` con el refresh token.
19. El backend verifica firma, tipo, caducidad y que el **hash SHA-256** exista y no esté revocado.
20. Si el token ya fue rotado (`revoked_at != null`), se **revocan todas las sesiones** del usuario y se audita `TOKEN_REUSE_DETECTED` con resultado `denied`.
21. Si es válido, se revoca el anterior y se emite un par nuevo.

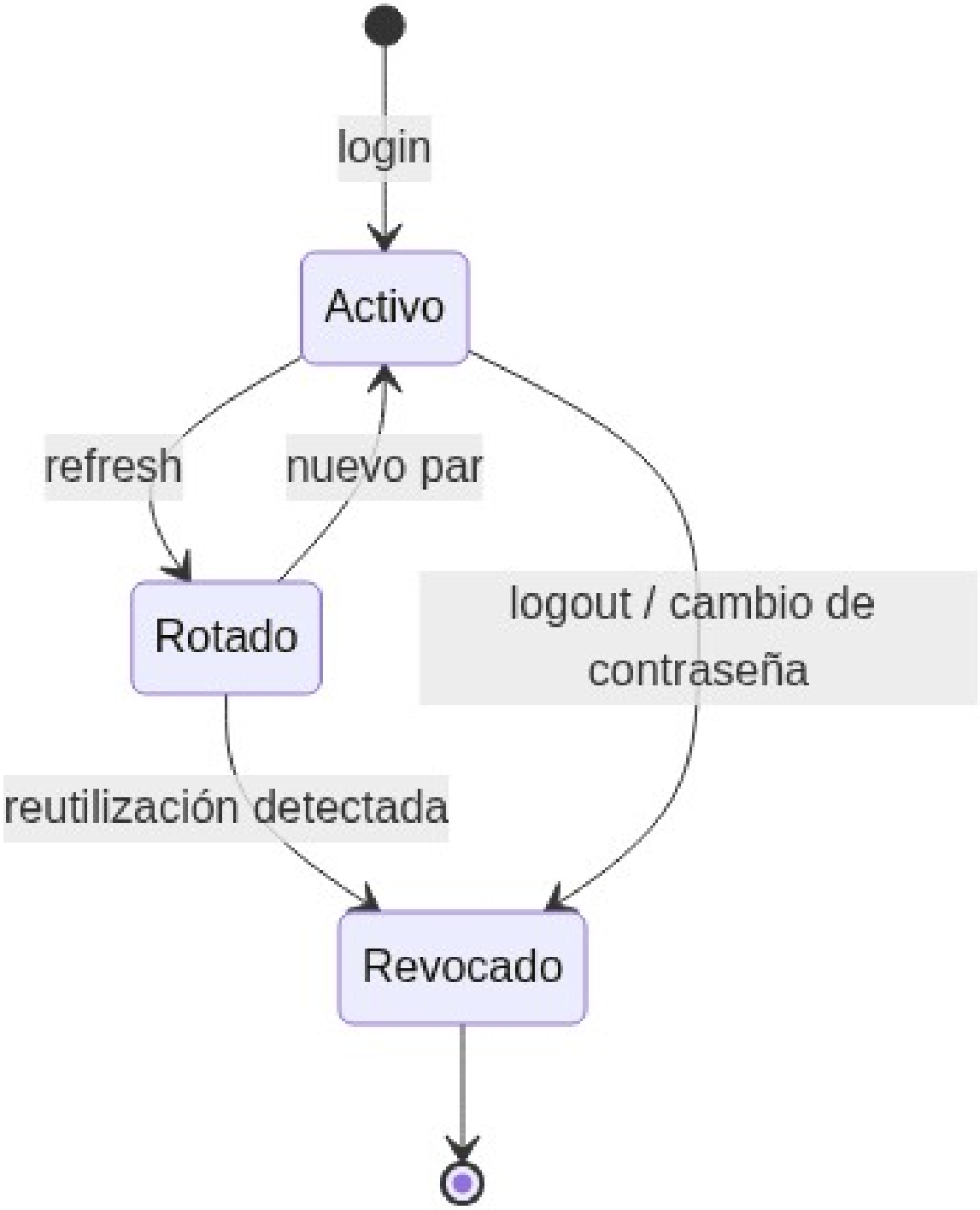


Figura 8. Diagrama de arquitectura / flujo

Contraseñas

- Hash **bcrypt coste 12** (bcrypt.hashpw), comparación con checkpw.
- Política: mínimo 10 caracteres, mayúscula, minúscula, dígito y símbolo; se valida también en creación y reseteo.
- POST /auth/change-password exige la contraseña actual, invalida todos los refresh tokens y audita el cambio.
- El administrador puede resetear con POST /users/{id}/reset-password (auditado, revoca sesiones).

Logout

POST /auth/logout revoca todos los refresh tokens del usuario (o sólo el enviado) y audita

LOGOUT. El access token sigue siendo válido hasta su expiración (30 min); para revocación inmediata se necesitaría una lista de denegación, documentada como evolución.

3.5 RBAC

Catálogo de permisos (26)

Módulo	Permisos
dashboard	read, export
alerts	read, create, update, delete, export
triage	read, run, feedback
incidents	read, create, update, close, export
users	read, create, update, delete
roles	read, update
audit	read, export
reports	read, create, export

Matriz rol ↔ permiso

Permiso	admin	soc_manager	analyst	auditor	viewer
dashboard:read	√	√	√	√	√
dashboard:export	√	√	—	—	—
alerts:read	√	√	√	√	√
alerts:create	√	√	√	—	—
alerts:update	√	√	√	—	—
alerts:delete	√	√	—	—	—
alerts:export	√	√	√	√	—
triage:read	√	√	√	√	—
triage:run	√	√	√	—	—
triage:feedback	√	√	√	—	—
incidents:read	√	√	√	√	√
incidents:create	√	√	√	—	—
incidents:update	√	√	√	—	—
incidents:close	√	√	—	—	—
incidents:export	√	√	—	—	—
users:read	√	√	—	—	—
users:create	√	—	—	—	—
users:update	√	—	—	—	—
users:delete	√	—	—	—	—
roles:read	√	√	—	—	—
roles:update	√	—	—	—	—
audit:read	√	√	—	√	—
audit:export	√	√	—	√	—

Permiso	admin	soc_manager	analyst	auditor	viewer
reports:read	✓	✓	✓	✓	✓
reports:create	✓	✓	✓	✓	—
reports:export	✓	✓	—	✓	—

El rol admin es **inmutable** desde la API para evitar el bloqueo del sistema; los demás roles se editan en caliente desde la pantalla de roles.

Implementación

```
# core/deps.py
def require_permission(*codes: str):
    async def checker(user: CurrentUser) -> User:
        if not any(user.has_permission(code) for code in codes):
            raise Forbidden(f"Se requiere uno de los permisos: {', '.join(codes)}")
        return user
    return checker

# routers/alerts.py
ReadAlerts = Annotated[User, Depends(require_permission(P.ALERTS_READ))]
```

El JWT incluye permisos para pintar el menú, pero get_current_user **relee rol y permisos de la base** en cada request. Un cambio de rol surte efecto en la siguiente petición, sin esperar a que expire el token.

Reglas de negocio asociadas

- No puede desactivarse a sí mismo.
- No puede eliminarse al último administrador activo.
- No puede retirar roles:update de su propio rol.
- El reset de contraseña revoca sesiones.
- Toda modificación de usuario/rol se audita con detail.changes.

3.6 Bitácora de auditoría

Doble registro

22. **Acceso (middleware):** para cada request a /api/v1, dos filas:
    - action=ENTER: método, ruta, query, usuario, IP, request-id, timestamp de inicio.
    - action=EXIT: status code, duración, mismo request-id.
23. **Negocio (servicios/routers):** eventos explícitos con contexto:

LOGIN, LOGIN_FAILED, TOKEN_REUSE_DETECTED, LOGOUT, PASSWORD_CHANGE, ALERT_INGEST, ALERT_DEDUP, ALERT_UPDATE, TRIAGE_RUN, TRIAGE_FEEDBACK, INCIDENT_CREATE, INCIDENT_UPDATE, USER_CREATE, USER_UPDATE, USER_DELETE, PASSWORD_RESET, ROLE_PERMISSIONS_UPDATE, REPORT_GENERATE, REPORT_DOWNLOAD, AUDIT_EXPORT.

Ejemplo de fila:

```
{
  "id": 142,
  "occurred_at": "2026-10-20T10:15:51.792Z",
  "username": "analista1",
  "module": "triage",
  "action": "TRIAGE_RUN",
```

```
 "entity_type": "alert",
 "entity_id": "d6d37274-...",
 "result": "success",
 "detail": {
  "verdict": "true_positive",
  "severity": "critical",
  "confidence": 0.92,
  "model": "llama3.1:8b",
  "fallback": false,
  "latency_ms": 14300
 },
 "ip": "10.10.1.30",
 "request_id": "46e1717c0e75466a81b5b0dec55780e4",
 "duration_ms": 14380,
 "hash": "dd71ef9a68d61a66..."
}
```


Inmutabilidad y cadena

- trg_audit_no_update bloquea UPDATE/DELETE.
- trg_audit_chain asigna chain_seq y calcula prev_hash/hash en cada INSERT bajo pg_advisory_xact_lock. La cadena se ordena por chain_seq, no por id, para que dos transacciones concurrentes no la rompan.
- GET /audit/verify-chain recalcula **en PostgreSQL** con la misma expresión del trigger y compara también el eslabón anterior. Devuelve {valid, checked, broken_at}.
- Exportación CSV/XLSX (/audit/export) también se audita.

Qué NO hace: no firma digitalmente, no envía a WORM y no protege contra un superusuario que desactive triggers. Se recomienda exportación periódica y permisos mínimos.

3.7 Ingesta y normalización

POST /api/v1/alerts/ingest acepta el JSON crudo de Wazuh/OPNsense/Suricata y lo normaliza.

Campo destino	Fuente Wazuh	Fallback
external_id	id	alert_id, event_id
rule_id	rule.id	rule_id, signature_id
rule_level	rule.level	rule_level, level
description	rule.description	message, full_log
src_ip	data.srcip	src_ip, source_ip
dst_ip	data.dstip	dst_ip, dest_ip
dst_port	data.dstport	dst_port
agent_name	agent.name	agent_name, host
asset_criticality	asset_criticality	agent.criticality, default 3
severity	severity explícita	Severity.from_rule_level()
tags	—	regex sobre descripción (bruteforce, ransomware, recon...)

Mapeo de severidad: level >= 12 → critical, >= 10 → high, >= 7 → medium, >= 4 → low, resto info. Una severidad explícita en el payload tiene prioridad.

Deduplicación: si el fingerprint ya existe, incrementa dedup_count, actualiza

last_seen_at y devuelve status=duplicate. El índice UNIQUE garantiza atomicidad incluso con dos webhooks concurrentes.

Respuesta:

```
{"status": "accepted", "alert_id": "...", "severity": "critical", "deduplicated": false,
 "message": "Alerta registrada en estado pendiente"}
```

Códigos: 202 aceptada o duplicada; 401 clave inválida; 422 payload inválido.

3.8 Triaje

POST /api/v1/alerts/{id}/triage (permiso triage:run):

24. Carga la alerta con enriquecimientos e historial.
25. Si existe un triaje de los últimos 60 s y force=false, lo devuelve (idempotencia).
26. Marca triaging y llama al microservicio LangChain (POST /triage, timeout 25 s).
27. Valida la respuesta con RemoteTriage (Pydantic).
28. Persiste triage_results, guarda enriquecimientos nuevos y actualiza la alerta:
    - false_positive → status=false_positive
    - true_positive con severidad alta/crítica → escalated
    - resto → triaged
    - la severidad de la alerta **nunca baja** automáticamente; sólo sube.
29. Audita TRIAGE_RUN con veredicto, modelo, latencia y si usó fallback.

Fallback local (services/triage_client.py): si el servicio no responde, timeout, JSON inválido o error 5xx, calcula un score con nivel de regla (55 %), criticidad (25 %) e indicadores maliciosos (20 %), marca status=degraded y fallback_used=true. Si TRIAGE_FAIL_OPEN=false, devuelve 503.

3.9 Incidentes

- POST /alerts/{id}/incident: crea ticket local OSK-######, prioridad derivada de la severidad (P1 crítica, P2 alta, P3 media, P4 baja), SLA 4/8/24/72 h y asigna al actor si no se indica otro.
- GET /incidents: filtros por estado, prioridad, responsable y texto; paginación.
- PATCH /incidents/{id}: estado, prioridad, responsable y resolución. Cerrar/resolver exige incidents:close; asigna closed_at automáticamente.
- La integración real con osTicket/Redmine la ejecuta n8n; el backend mantiene el registro local y el ticket_id para correlación.

3.10 Dashboard

Todas las métricas se calculan en PostgreSQL con ventanas parametrizadas (make_interval) y LATERAL para el último triaje.

Endpoint	Cálculo
/summary	Totales por severidad/estado, automatización, FP, SLA, MTTT y deltas vs. periodo anterior
/trend	generate_series diaria + conteos por severidad y FP
/severity	Distribución con denominador total
/top-sources	GROUP BY IP/agente/regla, orden descendente, límite

Endpoint	Cálculo
/mttt	AVG y PERCENTILE_CONT(0.95) por severidad + cumplimiento
/fp-rate	Serie diaria de triajes y FP
/sla	Objetivo por severidad y brechas
/analysts	Acciones auditadas y triajes por usuario

Definiciones:

- **MTTT:** triaged_at - received_at en minutos, usando el **último** triaje de cada alerta.
- **Automatización:** alertas con al menos un triaje / total.
- **Tasa FP:** veredicto false_positive / alertas triadas.
- **SLA:** porcentaje de triajes dentro del objetivo de su severidad.
- **Deltas:** periodo actual vs. periodo anterior de igual longitud.

3.11 Reportes

POST /api/v1/reports con {type, format, date_from, date_to}. El backend:

30. Crea la fila reports en estado generating y confirma (para tener ID).
31. Recolecta datos con las mismas funciones del dashboard + incidentes + alertas críticas + evolución mensual; para auditoría, agrega la bitácora y el resumen por módulo.
32. Renderiza en un hilo (asyncio.to_thread) para no bloquear el event loop:
    - **PDF:** ReportLab, A4, portada, KPIs, tabla de severidad con gráfico de barras, MTTT/SLA, top orígenes, alertas críticas, incidentes, evolución mensual y anexo de auditoría.
    - **DOCX:** python-docx con encabezados, tablas y metadatos.
    - **XLSX:** openpyxl con 8–10 hojas, estilos, autofiltro, panel congelado y gráficos nativos (línea de tendencia y barras de severidad).
33. Guarda en storage/reports/{uuid}.{ext}, actualiza estado, tamaño y completed_at.
34. Audita REPORT_GENERATE. La descarga audita REPORT_DOWNLOAD y valida que la ruta esté dentro del directorio permitido (anti path traversal).

Historial: GET /reports paginado; **descarga:** GET /reports/{id}/download con Content-Disposition y MIME correcto.

3.12 Contrato de errores

```
{
  "detail": "Se requiere uno de los permisos: users:read",
  "code": "FORBIDDEN",
  "request_id": "a0989711a417444a951e9e3377ef1731",
  "path": "/api/v1/users"
}
```

HTTP	code	Cuándo
400	WEAK_PASSWORD, UNKNOWN_PERMISSION	Validación de negocio
401	UNAUTHORIZED	Token ausente/expirado, credenciales inválidas
403	FORBIDDEN	Permiso insuficiente
404	NOT_FOUND	Recurso inexistente

HTTP	code	Cuándo
409	CONFLICT	Duplicado, última cuenta admin, incidente ya existente
422	VALIDATION_ERROR	Payload inválido (incluye errors[])
500	REPORT_FAILED	Fallo de generación
503	DB_UNAVAILABLE, TRIAGE_UNAVAILABLE	Dependencia caída

Los handlers globales capturan IntegrityError (409) y SQLAlchemyError (503), registran el request_id y nunca exponen el stack al cliente.

3.13 Pruebas

```
cd backend
.venv/bin/python -m pytest -q
# 14 passed in ~4 s
```

Archivo	Cobertura
test_auth.py	Login OK/fallido, perfil, RBAC, refresh con rotación y detección de reutilización
test_alerts.py	Ingesta sin clave, dedup, detalle, triaje con fallback, feedback, incidente, filtros
test_dashboard_reports.py	Summary y series, reporte XLSX + descarga, cadena de auditoría, evento n8n

Las pruebas usan **PostgreSQL real** (no mocks) y una fixture que dispone el pool asyncpg entre event loops. La base debe estar sembrada; los tests crean alertas con UUID para no colisionar. **Cobertura pendiente (honesta):** no hay pruebas unitarias del render PDF/DOCX (sólo XLSX), ni pruebas de carga, ni E2E de navegador automatizadas. Se documentan como siguiente paso.

3.14 Rendimiento y concurrencia

Tema	Decisión
Pool	pool_size=10, max_overflow=20, pool_pre_ping, reciclado 30 min
Consultas	Índices por severidad/estado/fecha; LATERAL para el último triaje; agregaciones en SQL
Paginación	LIMIT/OFFSET; para > 100 k filas se recomienda keyset (received_at, id)
Reportes	asyncio.to_thread evita bloquear el event loop
Auditoría	Advisory lock serializa la cadena; es el principal coste de escritura
Escalado	API sin estado → N réplicas detrás de Nginx; migrar reportes a cola (Celery/RQ) si crece

Cuello de botella esperado: el LLM (5–25 s por alerta). Estrategias: cachear veredictos por fingerprint, triar en paralelo con concurrencia limitada, usar un modelo 8B cuantizado y reservar el modelo grande para alta/crítica.

3.15 Cómo extender

Para añadir un módulo (p. ej. vulnerabilidades):

35. Crear tablas en db/schema.sql + enums y vista.
36. Añadir modelo en app/models/, esquema en app/schemas/.
37. Crear router en app/api/v1/routers/ y registrarlo en router.py.
38. Añadir permisos vulnerabilities:* en el catálogo y en seed.sql.
39. Añadir navegación y página en el frontend.
40. Añadir pruebas de integración.
41. Documentar el nuevo módulo y su mapeo normativo.

El sistema de auditoría, RBAC, reportes y dashboard son reutilizables sin cambios.

4. Desarrollo del frontend

> **Stack:** React 18.3 · TypeScript 5.6 (strict) · Vite 5.4 · Tailwind CSS 3.4 ·
>
> TanStack Query 5 · React Router 6 · Recharts 2 · Axios · lucide-react · date-fns.
>
> Código en `frontend/`.

4.1 Objetivo de la interfaz

La consola está diseñada para un analista que trabaja 8 horas mirando alertas. Por eso:

- **Jerarquía visual por severidad**: el color no es decorativo, es el primer canal de priorización (rojo crítico, naranja alto, amarillo medio, azul bajo, gris info).
- **Densidad controlada**: tablas compactas, tipografía monoespaciada para IPs/IDs, pero con aire suficiente para escanear.
- **Contexto antes que acción**: el detalle muestra el resumen del LLM, la evidencia y las acciones recomendadas antes del botón de validar.
- **Modo oscuro nativo**: reduce fatiga visual en turnos nocturnos; contraste AA en textos principales.
- **Estado siempre visible**: badges de estado, banner de cadena de auditoría, indicadores de "degradado" cuando el LLM no está disponible.

4.2 Estructura

```
frontend/
├── index.html
├── vite.config.ts          # proxy /api → :8010, chunking manual
├── tailwind.config.js      # paleta base/accent/severity, animaciones
├── tsconfig.json           # strict, noUnusedLocals, noUnusedParameters
├── src/
│   ├── main.tsx            # QueryClient + Router + Auth + Toast
│   ├── App.tsx             # rutas protegidas y por permiso
│   ├── index.css           # base, componentes (.card, .btn, .table…)
│   ├── api/
│   │   ├── client.ts       # axios, JWT, refresh automático, errores, descargas
│   │   └── index.ts        # endpoints tipados por módulo
│   ├── context/
│   │   ├── AuthContext.tsx    # sesión, login/logout, permisos
│   │   └── ToastContext.tsx   # notificaciones efímeras
│   ├── components/
```

```
│   │   ├── Layout.tsx        # sidebar, topbar, drawer móvil, reloj, menú de usuario
│   │   ├── RouteGuards.tsx   # ProtectedRoute, PermissionGate, AccessDenied
│   │   ├── Badges.tsx        # severidad, estado, veredicto, prioridad, confianza
│   │   └── ui.tsx            # PageHeader, KpiCard, Spinner, EmptyState,
│   │                         # Pagination, Modal, ErrorBanner
│   ├── pages/
│   │   ├── Login.tsx
│   │   ├── Dashboard.tsx
│   │   ├── Alerts.tsx
│   │   ├── AlertDetail.tsx
│   │   ├── Incidents.tsx
│   │   ├── Users.tsx
│   │   ├── Roles.tsx
│   │   ├── Audit.tsx
│   │   └── Reports.tsx
│   ├── lib/format.ts         # etiquetas, colores, fechas, duraciones, bytes
│   └── types/index.ts        # contrato TypeScript de la API
├── deploy/nginx.conf         # SPA + proxy + CSP
└── Dockerfile                # build multi-stage + Nginx
```

4.3 Rutas y control de acceso

Ruta	Página	Permiso	Guard
/login	Login	—	Redirige a / si ya hay sesión
/	Dashboard	dashboard:read	ProtectedRoute + PermissionGate
/alerts	Cola de alertas	alerts:read	idem
/alerts/:id	Detalle	alerts:read	idem
/incidents	Incidentes	incidents:read	idem
/users	Usuarios	users:read	idem
/roles	Matriz RBAC	roles:read	idem
/audit	Bitácora	audit:read	idem
/reports	Reportes	reports:read	idem
*	404	—	—

```tsx
// components/RouteGuards.tsx
export function ProtectedRoute({ children }: { children: ReactNode }) {
  const { user, loading } = useAuth()
  const location = useLocation()
  if (loading) return <Spinner label="Validando sesión…" />
  if (!user) return <Navigate to="/login" replace state={{ from: location.pathname }} />
  return <>{children}</>
}

export function PermissionGate({ permission, children, fallback = null }) {
  const { hasPermission } = useAuth()
  return hasPermission(permission) ? <>{children}</> : <>{fallback}</>
}
```

La UI **oculta** lo que el rol no puede hacer, pero la decisión real está en el backend. Si un usuario fuerza la ruta, ve AccessDenied y el intento queda en la bitácora.

4.4 Sesión y cliente HTTP

Login

42. POST /auth/login → {access_token, refresh_token, user}.
43. Se guardan los tokens en localStorage (soclite.access_token, soclite.refresh_token).
44. AuthProvider carga GET /auth/me al montar para validar la sesión y refrescar permisos.

Refresh transparente

```
api.interceptors.response.use(
 (response) => response,
 async (error: AxiosError) => {
  const config = error.config as RetryableConfig | undefined
  if (error.response?.status === 401 && config && !config._retry && !config.url?.includes('/auth/')) {
   config._retry = true
   refreshPromise = refreshPromise ?? refreshAccessToken()
   const token = await refreshPromise
   refreshPromise = null
   if (token) {
    config.headers = { ...config.headers, Authorization: `Bearer ${token}` }
    return api.request(config)
   }
  }
  return Promise.reject(error)
 },
)
```

- Si el refresh falla, se limpia el almacenamiento, se emite soclite:logout y el AuthProvider redirige al login.
- Se comparte una única promesa de refresh para evitar tormentas de 401 con varias queries concurrentes.
- Errores homogéneos con apiErrorMessage(): usa detail del backend, errores de validación o mensajes de red/TimeOut.

Descargas

downloadBlob() crea un object URL, dispara un <a download> y libera el recurso. Se usa para reportes y exportación de auditoría.

4.5 Datos y estado

TanStack Query maneja el estado remoto; no hay Redux ni stores globales innecesarios.

Query key	Contenido	Invalidación
['dashboard', days, ...]	summary, trend, severity, sources, mttt, fp	tras triaje, feedback, incidente
['alerts', filters]	página/filtros de alertas	tras mutaciones de alerta
['alert', id]	detalle completo	tras triaje/feedback/estado/incidente
['incidents', filters]	lista de incidentes	tras actualizar incidente
['users', filters], ['roles'], ['permissions']	administración	tras crear/editar/desactivar
['audit', filters], ['audit-modules']	bitácora	manual / tras verificar cadena
['reports', page], ['report-types']	historial y tipos	tras generar

- staleTime: 30 s, retry: 1, sin refetch al enfocar la ventana (evita parpadeos).
- placeholderData: previous en listas paginadas para mantener la tabla mientras llega la nueva

página.

- Los filtros viven en el estado local de cada página; la URL no se sincroniza (mejora futura: query params para compartir vistas).

4.6 Sistema de diseño

Paleta

Token	Valor	Uso
base-950	#050A13	Fondo
base-900	#080F1C	Tarjetas
base-850	#0B1524	Superficies elevadas
base-700	#1A2942	Bordes
accent	#22D3EE	Acción primaria, foco, datos
critical	#FB7185	Severidad crítica
high	#FB923C	Alta
medium	#FACC15	Media
low	#38BDF8	Baja
info	#94A3B8	Informativa

Componentes base (index.css)

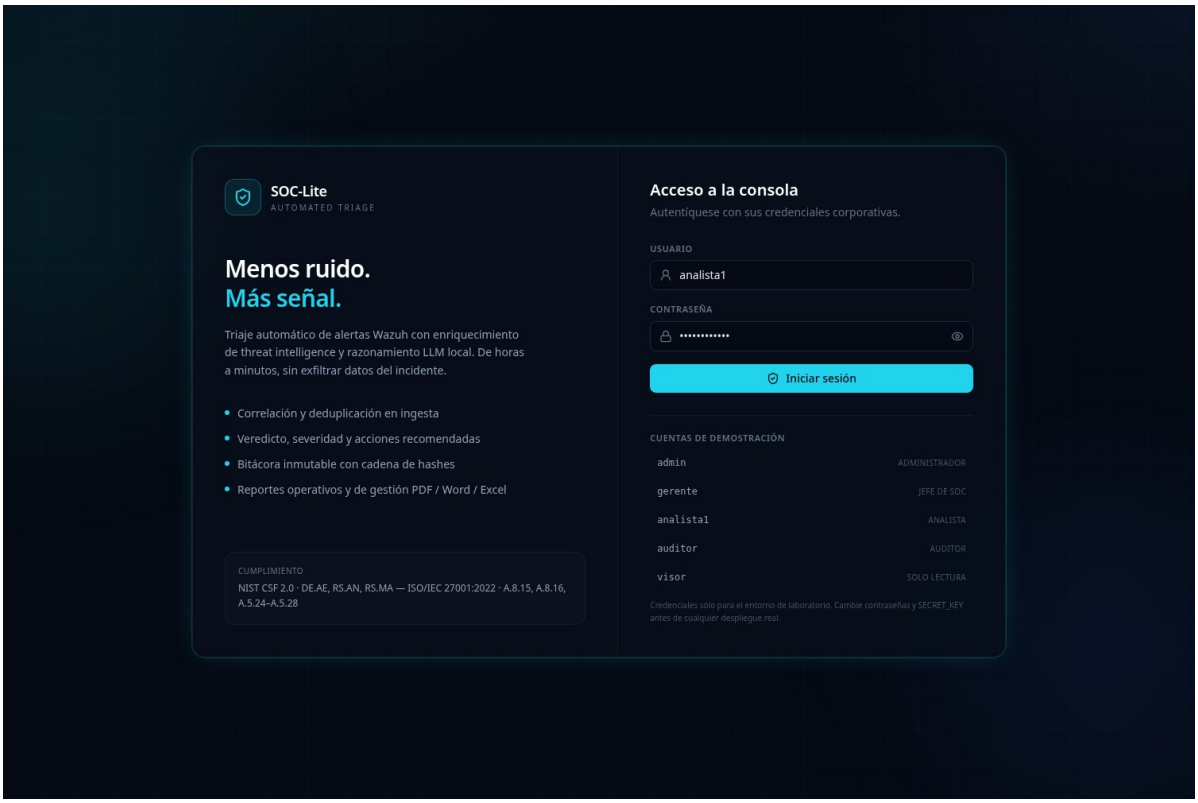
.card, .card-hover, .input, .btn-primary, .btn-secondary, .btn-ghost, .btn-danger, .table, .chip, .kpi-value, .label, .skeleton. Todos con foco visible y estados disabled.

Badges semánticos

SeverityBadge, StatusBadge, VerdictBadge, PriorityBadge y Confidence traducen enums del backend a color + etiqueta en español, con un punto de color para daltonismo parcial (no sólo color).

4.7 Páginas

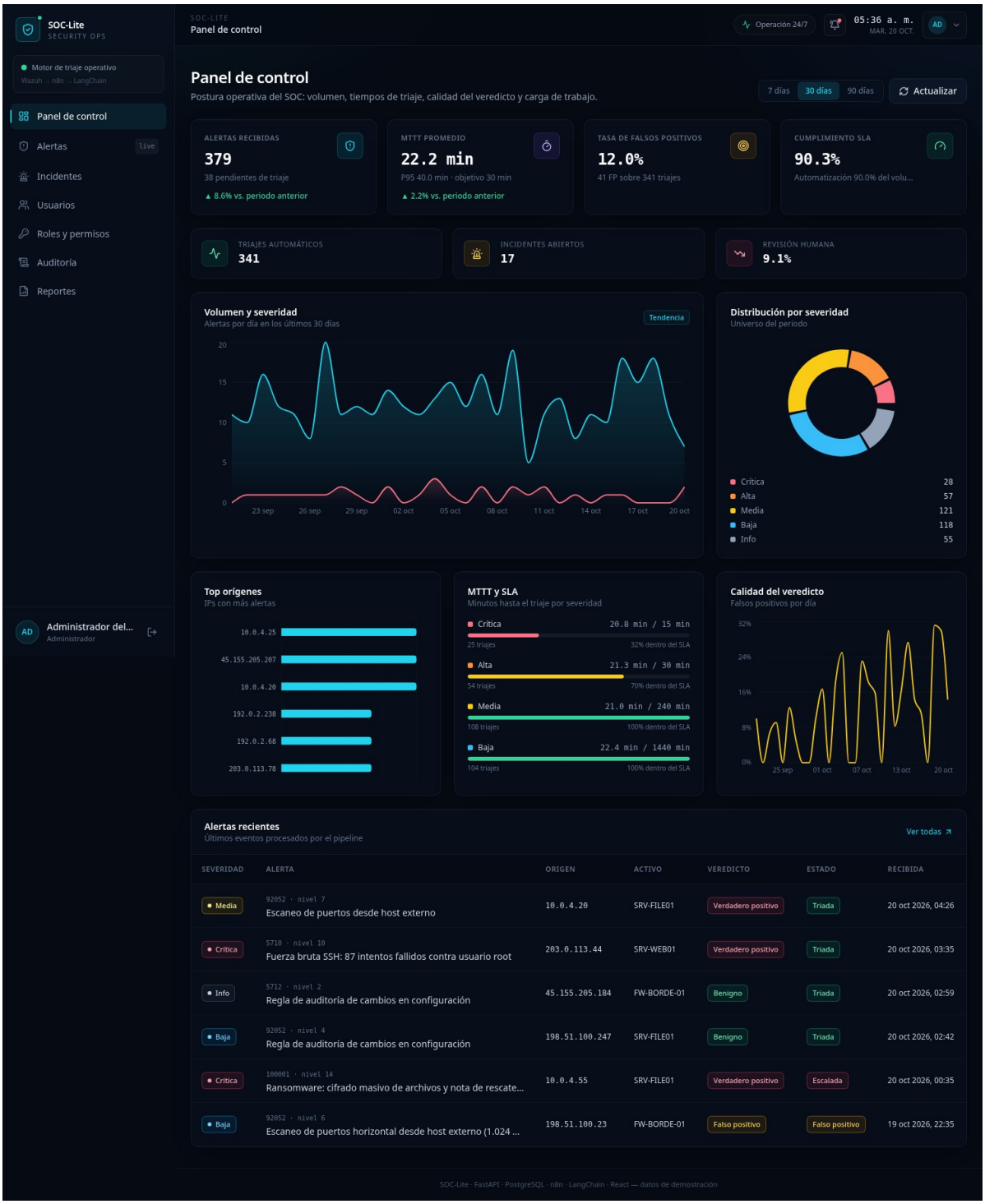
4.7.1 Login



Login

- Panel izquierdo con propuesta de valor, cumplimiento y estado del motor.
- Formulario con usuario/contraseña, mostrar/ocultar, error inline y estado de carga.
- **Cuentas de demostración** clicables para el laboratorio (admin, gerente, analista, auditor, visor).
- Redirección al from original tras autenticarse.

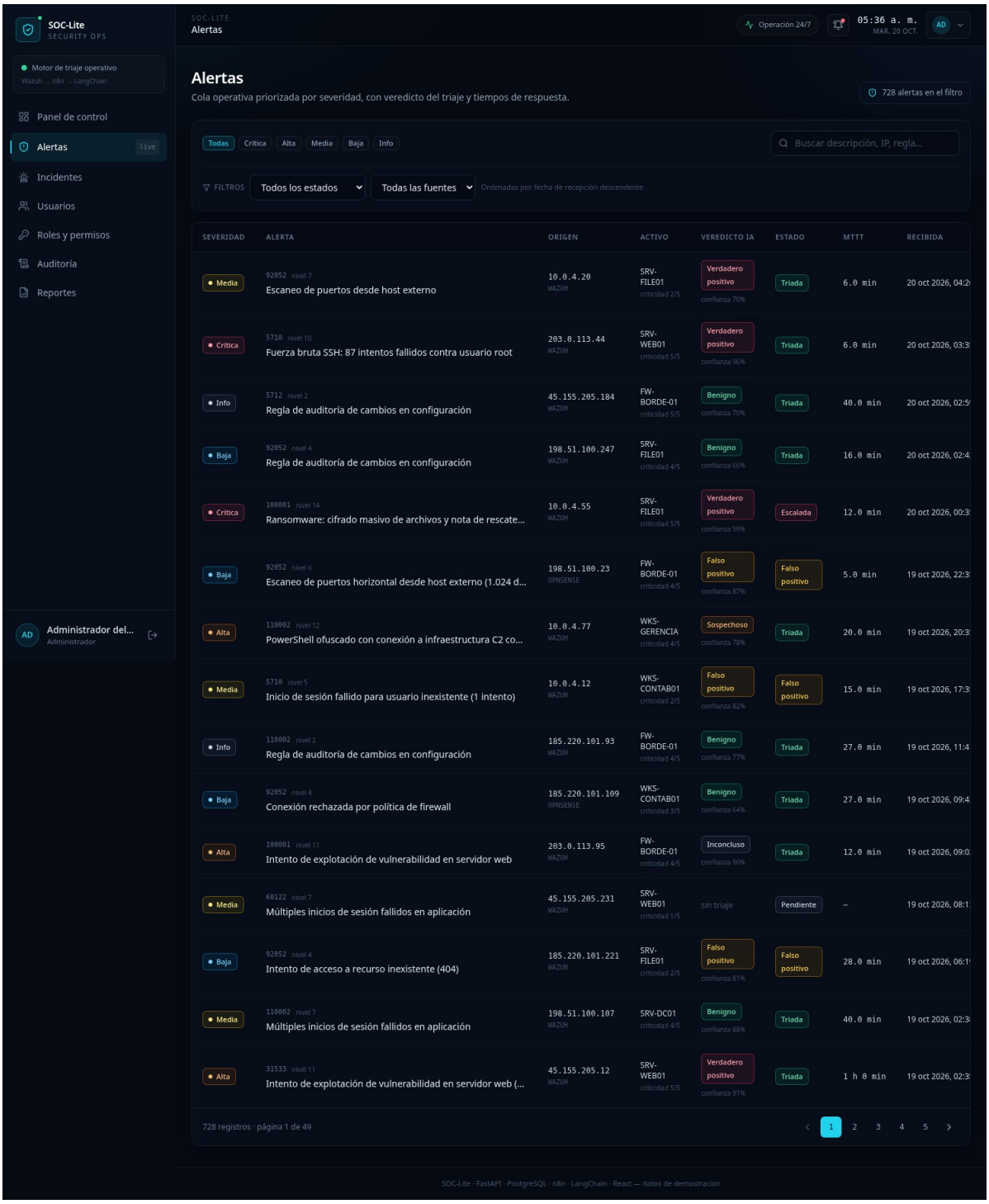
4.7.2 Dashboard



Dashboard

- Selector de periodo 7/30/90 días y botón de actualización.
- 4 KPIs principales: alertas, MTTT, tasa FP, SLA. Deltas vs. periodo anterior.
- 3 KPIs secundarios: triajes automáticos, incidentes abiertos, revisión humana.
- Gráfico de área (volumen total y críticas), dona de severidad, barras de top orígenes, línea de FP diaria.
- Panel de MTTT/SLA por severidad con barra de cumplimiento.
- Tabla de alertas recientes con enlace al detalle.

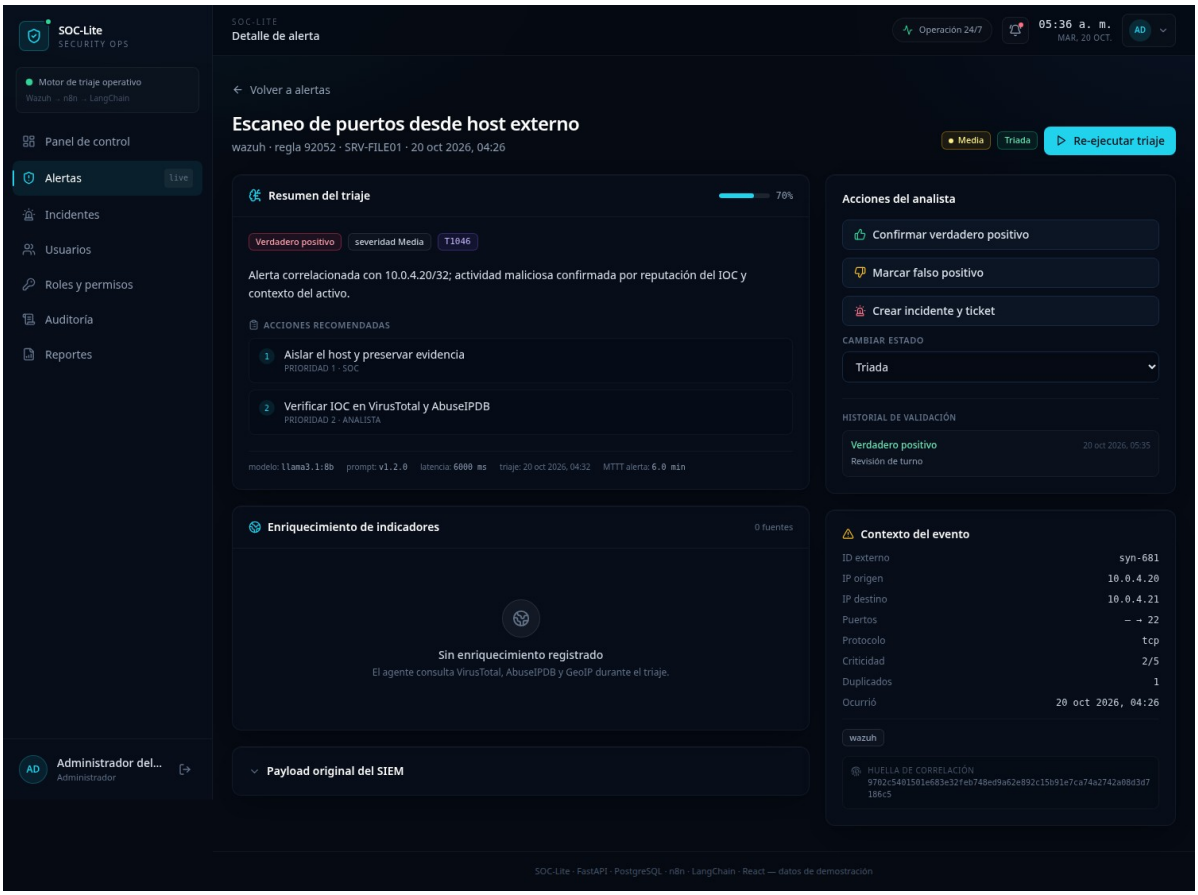
4.7.3 Alertas



Alertas

- Chips de severidad, búsqueda con debounce (350 ms), filtros de estado y fuente.
- Tabla con regla/nivel, descripción, IP, activo y criticidad, veredicto + confianza, estado, MTTT y fecha.
- Indicador ×N duplicados cuando la deduplicación agregó eventos.
- Paginación con total y páginas; placeholderData evita el salto visual.

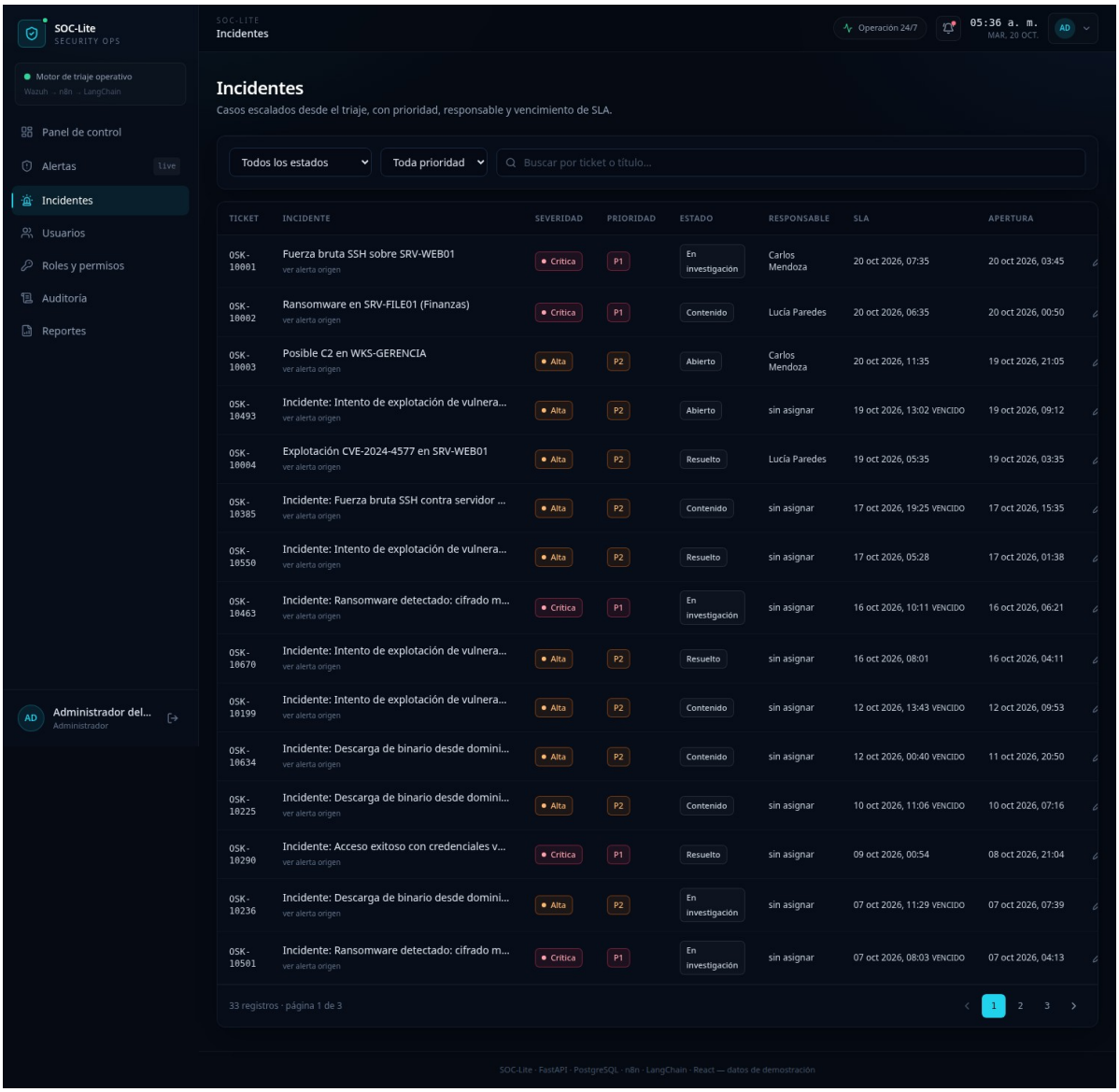
4.7.4 Detalle de alerta



Detalle

- Cabecera con severidad, estado y acción de (re)triaje.
- Tarjeta de triaje: veredicto, severidad sugerida, confianza, resumen, razonamiento, acciones numeradas, MITRE, modelo/prompt/latencia y aviso de modo degradado.
- Tabla de enriquecimiento por proveedor (VirusTotal, AbuseIPDB, GeoIP).
- Payload original del SIEM en un <details> (no satura la vista).
- Panel derecho: confirmar TP / marcar FP (con comentario), crear incidente, cambiar estado, historial de validaciones, contexto del evento, huella y ticket vinculado.
- Cada mutación invalida alert, alerts y dashboard.

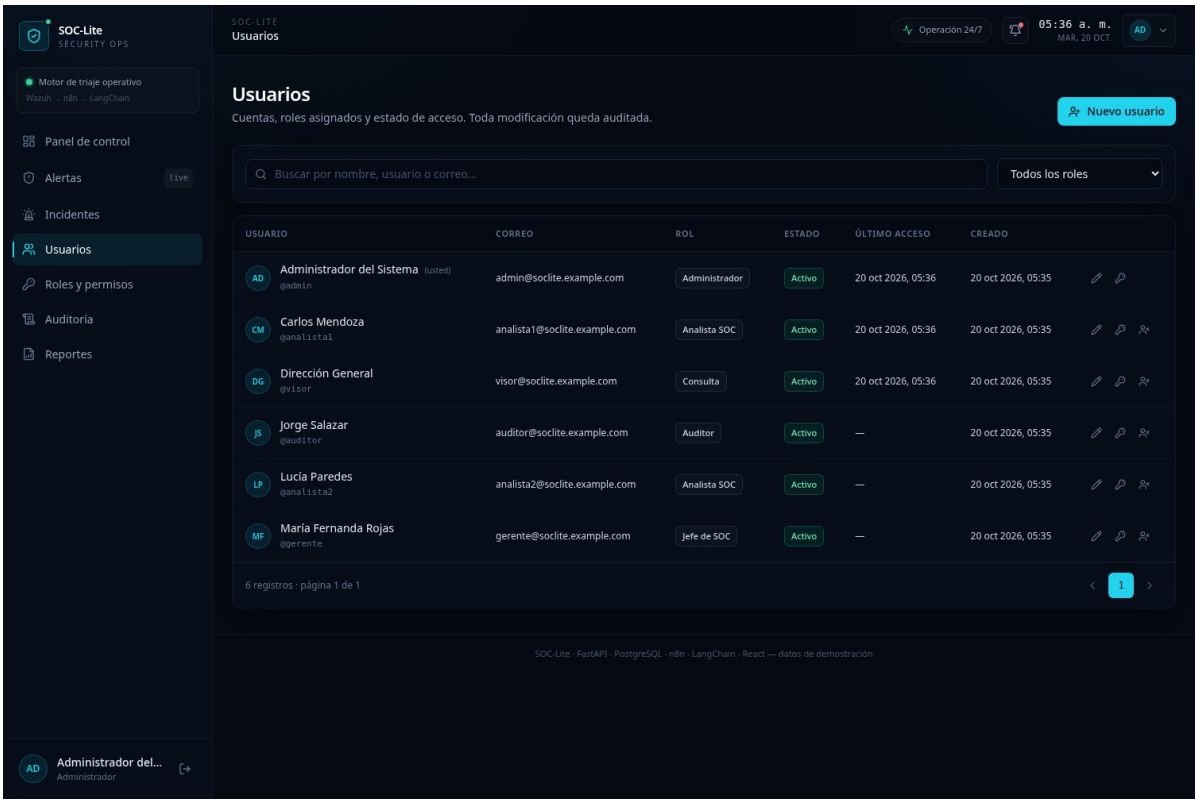
4.7.5 Incidentes



Incidentes

- Filtros por estado/prioridad y búsqueda por ticket o título.
- Tabla con severidad de la alerta, prioridad, estado, responsable, SLA (resaltado si vencido) y enlace a la alerta origen.
- Modal de edición con estado, prioridad, resolución y notas; cierre auditado.

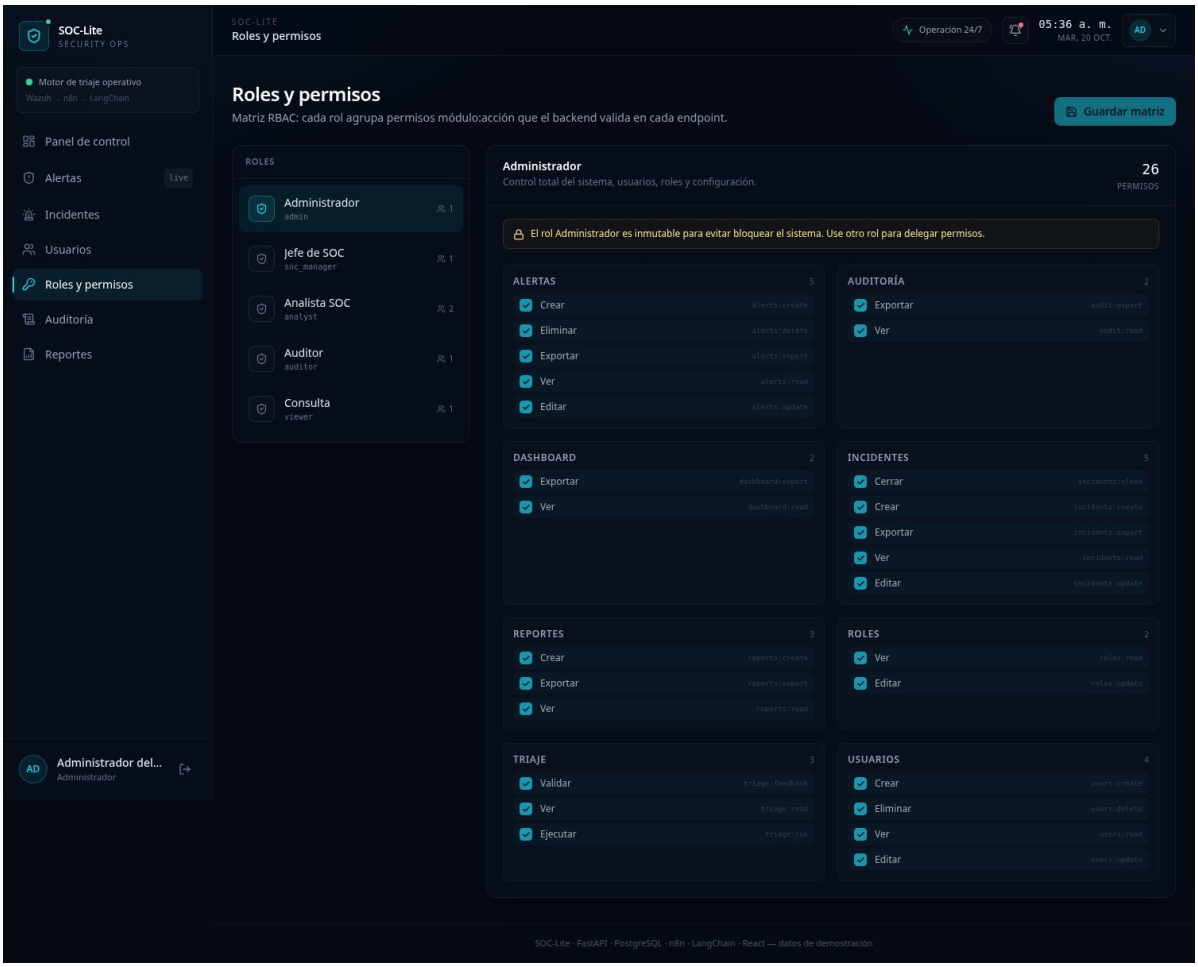
4.7.6 Usuarios



Usuarios

- Búsqueda y filtro por rol; alta/edición en modal; reset de contraseña; baja lógica.
- Muestra "usted" para la propia cuenta y bloquea auto-desactivación.
- Roles cargados desde /roles; el selector refleja el RBAC real.

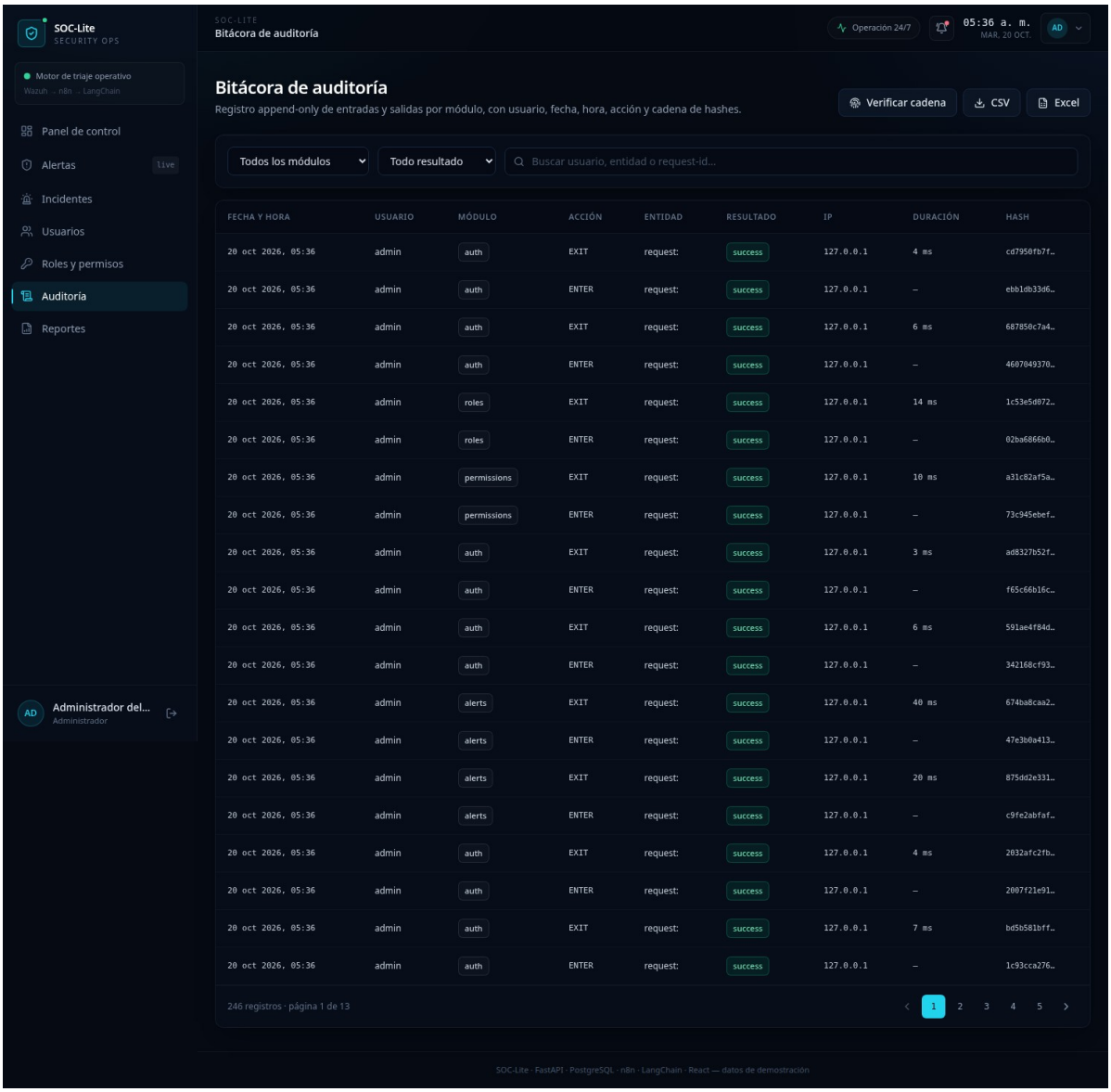
4.7.7 Roles y permisos



Roles

- Lista de roles con número de usuarios.
- Matriz agrupada por módulo con checkboxes; contador de permisos y estado "dirty".
- El rol Administrador se muestra inmutable con candado.
- Guardado con PUT /roles/{id}/permissions; invalida roles y permisos.

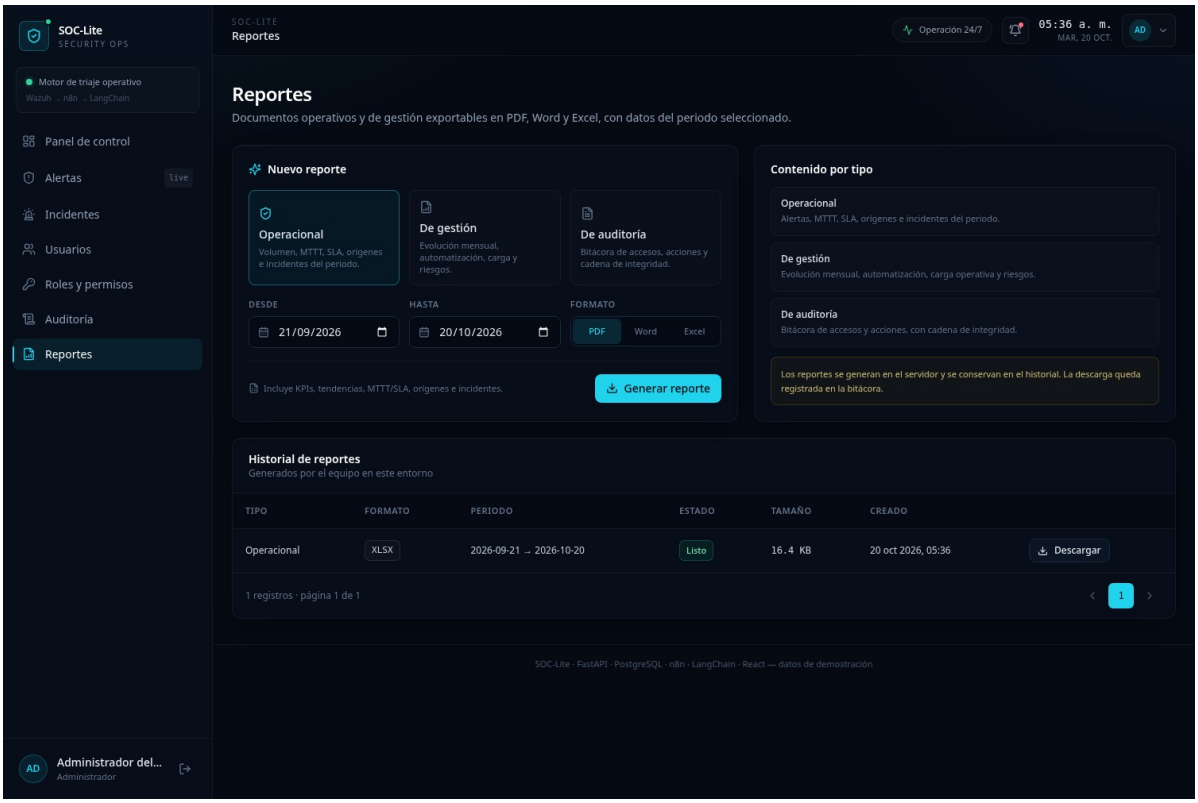
4.7.8 Auditoría



Auditoría

- Filtros por módulo, resultado y búsqueda por usuario/entidad/request-id.
- Botón **Verificar cadena** → banner verde/rojo con eventos verificados.
- Exportación CSV/Excel con descarga directa.
- Tabla con fecha/hora, usuario, módulo, acción, entidad, resultado, IP, duración y hash corto.
- Modal de detalle con JSON y hash completo.

4.7.9 Reportes



Reportes

- Tarjetas de tipo (operacional, gestión, auditoría), rango de fechas y selector PDF/Word/Excel.
- Generación con estado de carga; el backend responde cuando el archivo está listo.
- Historial con tipo, formato, periodo, estado, tamaño y descarga.
- Aviso de que la descarga queda auditada.

4.8 Responsive y accesibilidad

- **Breakpoints:** sidebar fija en lg (≥1024 px); drawer con overlay en móvil; tablas con scroll horizontal y columnas mínimas.
- **Foco:** focus:ring-2 en inputs, botones y enlaces; navegación por teclado.
- **Contraste:** textos principales slate-200 sobre base-950; severidad siempre con etiqueta textual, no sólo color.
- **Estados:** aria-label en botones icónicos, role="status" en toasts, aria-hidden implícito en iconos decorativos.
- **Formularios:** labels asociados por htmlFor, autoComplete en login, mensajes de error visibles.
- **Movimiento reducido:** las animaciones son sutiles (fade-up); no hay parallax ni auto-scroll.
- **Pendiente:** auditoría formal con axe/Lighthouse y pruebas con lector de pantalla.

4.9 Build y rendimiento

```
npm run typecheck   # tsc --noEmit → 0 errores
npm run build       # tsc -b && vite build
```

Resultado medido:

Artefacto	Tamaño	Gzip
index.html	0.87 kB	0.42 kB
index.css	34.0 kB	6.6 kB
index.js	178.9 kB	51.3 kB
vendor.js (React/Router)	164.0 kB	53.5 kB
charts.js (Recharts)	423.1 kB	113.0 kB
query.js	41.6 kB	12.5 kB
icons.js	24.7 kB	5.1 kB
Total	**~867 kB**	**~242 kB**

Decisiones de rendimiento

- manualChunks separa vendor/charts/query/icons: el dashboard cachea Recharts y las páginas no lo vuelven a descargar.
- Sin librería de componentes pesada (MUI/AntD): Tailwind + componentes propios.
- Iconos tree-shakeados por import individual.
- Imágenes: ninguna en la app (todo CSS/SVG), por lo que no hay LCP de imagen.
- React.lazy no se usó porque el bundle inicial es razonable; se puede adoptar si se añaden más módulos.

4.10 Configuración y despliegue

Variable	Default	Uso
VITE_API_URL	/api/v1	Base de la API; en Docker se proxya con Nginx

- **Desarrollo:** Vite en 8090 con proxy /api → http://127.0.0.1:8010.
- **Producción:** build estático + Nginx en 8090; /api/ proxya a backend:8000; CSP estricta; try_files para SPA; cache inmutable para assets con hash.
- **Docker:** build multi-stage (node:22-alpine → nginx:1.27-alpine), healthcheck con wget.

4.11 Checklist de verificación manual

- [x] Login con cada uno de los 6 usuarios demo.
- [x] Menú filtrado por permisos (visor no ve usuarios/roles/auditoría).
- [x] Dashboard con 7/30/90 días; tooltips y series.
- [x] Filtros de alertas, búsqueda, paginación y detalle.
- [x] Triaje manual, feedback FP/TP, creación de incidente y cambio de estado.
- [x] Matriz de roles con guardado y rol admin bloqueado.
- [x] Verificación de cadena de auditoría y exportación CSV.
- [x] Generación y descarga de PDF/Word/Excel.
- [x] Layout responsive (drawer móvil, tablas con scroll).
- [x] Sin errores de consola salvo favicon.ico 404 (no crítico).

Pendiente: pruebas E2E automatizadas (Playwright) y auditoría de accesibilidad formal.

5. Flujo de nodos n8n

> Workflow importable: `n8n/soc_lite_triage_workflow.json`
>
> · 21 nodos (10 operativos + soporte) · importado y validado en n8nio/n8n:latest.

5.1 Por qué n8n

Necesidad	Cómo la cubre n8n
Integrar Wazuh, OPNsense, PostgreSQL, HTTP, Slack y osTicket	Nodos nativos + HTTP Request
Reintentos y manejo de errores	retryOnFail, maxTries, salidas de error, onError
Visibilidad para operación	Ejecuciones, items de entrada/salida y editor visual
Cambios sin desplegar código	Versionado del JSON + importación
Idempotencia	ON CONFLICT en PostgreSQL + fingerprint
Bajo coste	Open source, autoalojado

n8n **no** clasifica: sólo orquesta. El razonamiento vive en LangChain y la validación/persistencia de negocio en FastAPI.

5.2 Los 10 nodos requeridos y su implementación

#	Nodo (diseño)	Nodo real	Tipo	Función	Entrada	Salida
1	Webhook	**Webhook SIEM**	webhook	Recibe Wazuh/OPNsense	JSON crudo	body normalizable
2	IF	**Severidad minima + Duplicada**	if (2 nodos)	Descarta < medium; detecta duplicado	severity, deduplicated	Rama true/false
3	PostgreSQL	**Persistir alerta**	postgres	Upsert con fingerprint	payload normalizado	id, severity, deduplicated
4	HTTP Request	**Triaje LangChain**	httpRequest	Llama al microservicio	alerta normalizada	JSON de triaje
5	Agente LangChain	**Microservicio `/triage`**	servicio externo	Enriquece + clasifica	alerta + tools	verdict, severity, actions
6	Switch	**Switch severidad**	switch	Enruta por severidad	severity	3 salidas
7	PostgreSQL	**Guardar triaje**	postgres	Inserta triaje y actualiza alerta	JSON del agente	alert_id, status
8	Slack/Telegram	**Slack**	httpRequest	Notifica sólo alta/crítica	resumen	mensaje
9	HTTP Request	**osTicket**	httpRequest	Crea ticket	triaje	ticket_id
10	HTTP Request	**Auditoria**	httpRequest	Registra en bitácora	acción	hash

Nodos de soporte

Nodo	Tipo	Para qué
Normalizar alerta	code	Mapea payload, calcula severidad, tags y fingerprint
Preparar guardado	code	Empaqueta el triaje en base64 (un solo parámetro SQL)
Notificar	if	Sólo high/critical
Marcar pendiente	postgres	Si LangChain falla, deja la alerta pending
Auditoria error	httpRequest	Audita el fallo de triaje
Responder OK / duplicada / descartada / error	respondToWebhook	Respuesta HTTP al SIEM
Nota de arquitectura / enrutamiento	stickyNote	Documentación dentro del lienzo

5.3 Grafo del workflow

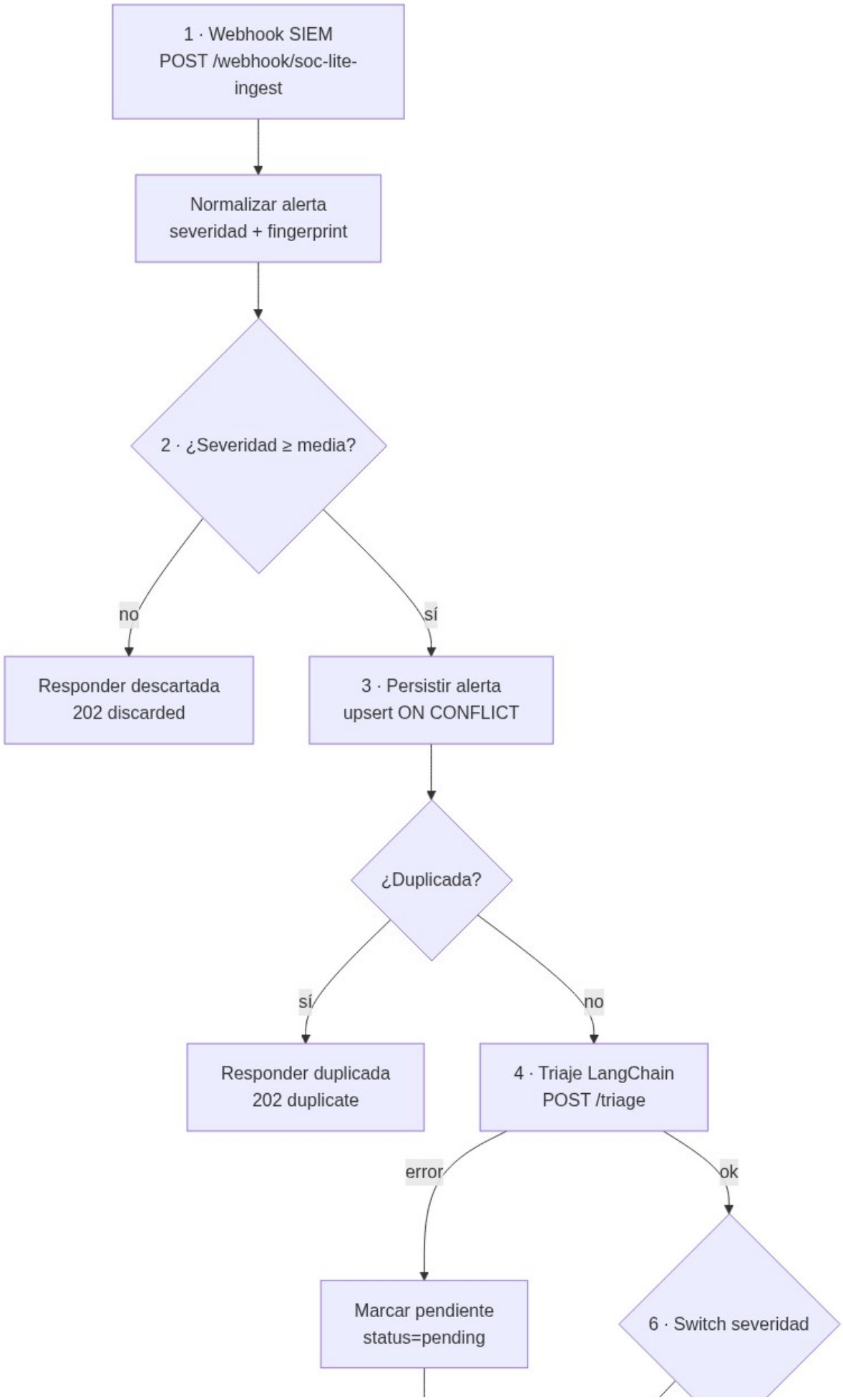

1 · Webhook SIEM
POST /webhook/soc-lite-ingest
Normalizar alerta
severidad + fingerprint
2 · ¿Severidad ≥ media?
no
sí
Responder descartada
202 discarded
3 · Persistir alerta
upsert ON CONFLICT
¿Duplicada?
sí
no
Responder duplicada
202 duplicate
4 · Triaje LangChain
POST /triage
error
ok
Marcar pendiente
status=pending
6 · Switch severidad

Figura 9. Diagrama de arquitectura / flujo

5.4 Paso a paso

Nodo 1 · Webhook SIEM

- **Tipo:** n8n-nodes-base.webhook v2.
- **Configuración:** httpMethod=POST, path=soc-lite-ingest, responseMode=responseNode.
- **Entrada:** JSON de Wazuh (rule, agent, data) o de OPNsense.
- **Salida:** item con body.
- **Seguridad:** el webhook queda en la red interna; en producción se publica detrás de Nginx con

TLS y allowlist de IPs del SIEM.

```
{
 "parameters": {
  "httpMethod": "POST",
  "path": "soc-lite-ingest",
  "responseMode": "responseNode",
  "options": { "rawBody": false }
 },
 "type": "n8n-nodes-base.webhook",
 "typeVersion": 2
}
```

Nodo 2 · Normalizar alerta (Code)

Extrae los campos comunes, calcula severidad y construye una huella SHA-256 en ventana de 15 min.

```
const body = $json.body ?? $json;
const rule = body.rule ?? {};
const data = body.data ?? {};
const level = Number(rule.level ?? 0);

let severity = body.severity;
if (!severity) severity = level >= 12 ? 'critical' : level >= 10 ? 'high'
 : level >= 7 ? 'medium' : level >= 4 ? 'low' : 'info';

const srcIp = data.srcip ?? body.src_ip ?? null;
const srcIpNorm = srcIp ?? '';
const fingerprint = require('crypto').createHash('sha256')
 .update([source, ruleId, srcIpNorm, dstIp ?? '', externalId, window].join('|'))
 .digest('hex');

return [{ json: { ...record, passes_filter: ['critical','high','medium'].includes(severity),
            payload_b64: Buffer.from(JSON.stringify(record)).toString('base64') } }];
```

- **Salida:** severity, tags (pipe-separated), fingerprint, payload_b64, passes_filter.

Nodo 3 · ¿Severidad ≥ media? (IF)

- **Condiciones (OR):** severity == critical, high, medium.
- **True:** persiste y tria.
- **False:** responde 202 discarded sin tocar la base (ahorro de almacenamiento y ruido).

```
{
 "conditions": {
  "options": { "caseSensitive": true, "typeValidation": "loose", "version": 2 },
  "conditions": [
   { "leftValue": "={{ $json.severity }}", "rightValue": "critical",
     "operator": { "type": "string", "operation": "equals" } },
   { "leftValue": "={{ $json.severity }}", "rightValue": "high",
     "operator": { "type": "string", "operation": "equals" } },
   { "leftValue": "={{ $json.severity }}", "rightValue": "medium",
```

```
      "operator": { "type": "string", "operation": "equals" } }
    ],
    "combinator": "or"
  }
}
```

Nodo 4 · Persistir alerta (PostgreSQL)

Un solo parámetro base64 evita problemas de comas en descripciones y payloads.

```
INSERT INTO alerts (
  external_id, source, rule_id, rule_level, description, src_ip, dst_ip, dst_port,
  protocol, agent_name, asset_criticality, severity, status, fingerprint, raw_payload,
  tags, occurred_at, received_at, last_seen_at
)
SELECT
  NULLIF(x.external_id,''), x.source, NULLIF(x.rule_id,''), x.rule_level, x.description,
  NULLIF(x.src_ip,'')::inet, NULLIF(x.dst_ip,'')::inet, x.dst_port, x.protocol,
  x.agent_name, COALESCE(x.asset_criticality,3), x.severity::severity_level, 'pending',
  x.fingerprint, x.raw_payload, string_to_array(COALESCE(x.tags,''),'|'),
  COALESCE(NULLIF(x.occurred_at,'')::timestamptz, now()), now(), now()
FROM jsonb_to_record(convert_from(decode($1,'base64'),'UTF8')::jsonb) AS x(
  external_id text, source text, rule_id text, rule_level smallint, description text,
  src_ip text, dst_ip text, dst_port integer, protocol text, agent_name text,
  asset_criticality smallint, severity text, fingerprint text, raw_payload jsonb,
  tags text, occurred_at text
)
ON CONFLICT (fingerprint) DO UPDATE
  SET dedup_count = alerts.dedup_count + 1,
    last_seen_at = now()
RETURNING id, severity, (xmax <> 0) AS deduplicated;
```

- **Salida:** id, severity, deduplicated.
- **Idempotencia:** ON CONFLICT atómico; dos webhooks simultáneos no crean duplicados.
- **`xmax <> 0`:** detecta si fue UPDATE (duplicado) o INSERT (nuevo).

Nodo 5 · ¿Duplicada? (IF)

- **Condición:** deduplicated == true.
- **True:** responde 202 duplicate; no se vuelve a triar (evita coste de LLM).
- **False:** llama al triaje.

Nodo 6 · Triaje LangChain (HTTP Request)

- **Método:** POST.
- **URL:** {{$env.LANGCHAIN_URL}}/triage (default http://triage:8001/triage).
- **Body:** alerta normalizada + alert_id de PostgreSQL.
- **Reintentos:** 2 intentos, 2 s de espera.
- **Timeout:** 60 s.
- **Salida OK:** verdict, severity, confidence, summary, recommended_actions, mitre_techniques, model, prompt_version, latency_ms, status, fallback_used.
- **Salida error:** conectada a **Marcar pendiente**.

```
{
  "method": "POST",
  "url": "={{ $env.LANGCHAIN_URL || 'http://langchain:8001' }}/triage",
  "sendBody": true,
  "specifyBody": "json",
        "jsonBody": "={{ JSON.stringify({ alert_id: $('Persistir alerta').item.json.id, description: $('Normalizar alerta').item.json.description, src_ip: $('Normalizar alerta').item.json.src_ip, severity: $('Normalizar alerta').item.json.severity }) }}",
  "options": { "timeout": 60000 }
}
```

Nodo 7 · Agente LangChain (servicio)

No es un nodo de n8n: es el microservicio el que ejecuta el agente con tool calling. n8n sólo ve el contrato HTTP. Si el servicio está caído, n8n no inventa un veredicto: marca la alerta pendiente.

Nodo 8 · Switch severidad

- **Tipo:** switch v3, tres salidas nombradas.
- **Regla 1 (Alta/Crítica):** severity == critical OR high.
- **Regla 2 (Media):** severity == medium.
- **Regla 3 (Baja/Info):** severity == low OR info.
- Las tres salidas convergen en **Preparar guardado**; el enrutamiento determina notificación, no persistencia.

Nodo 9 · Preparar guardado (Code)

Empaqueta el triaje en JSON base64 y conserva alert_id, severity, verdict para los nodos posteriores.

```
const t = $json;
const alertId = $('Persistir alerta').item.json.id;
const payload = {
 alert_id: alertId,
 verdict: t.verdict ?? 'inconclusive',
 severity: t.severity ?? 'medium',
 confidence: Number(t.confidence ?? 0.5),
 summary: String(t.summary ?? 'Triaje sin resumen').slice(0, 1200),
 reasoning: t.reasoning ?? null,
 recommended_actions: t.recommended_actions ?? [],
 mitre_techniques: t.mitre_techniques ?? [],
 model: t.model ?? 'langchain',
 prompt_version: t.prompt_version ?? 'v1.2.0',
 latency_ms: Number(t.latency_ms ?? 0),
 status: t.status ?? 'completed',
 needs_human: Boolean(t.needs_human),
 fallback_used: Boolean(t.fallback_used),
 raw_response: t,
};
return [{ json: { alert_id: alertId, severity: payload.severity, verdict: payload.verdict,
 summary: payload.summary, status: payload.status,
 payload_b64: Buffer.from(JSON.stringify(payload)).toString('base64') } }];
```

Nodo 10 · Guardar triaje (PostgreSQL)

CTE atómica: inserta el triaje y actualiza la alerta en una sola sentencia.

```
WITH payload AS (
  SELECT convert_from(decode($1,'base64'),'UTF8')::jsonb AS j
), ins AS (
  INSERT INTO triage_results (
    alert_id, verdict, severity, confidence, summary, reasoning, recommended_actions,
    mitre_techniques, model, prompt_version, latency_ms, status, needs_human,
    fallback_used, raw_response, triaged_at
  )
  SELECT
    (j->>'alert_id')::uuid, (j->>'verdict')::triage_verdict,
    (j->>'severity')::severity_level, COALESCE((j->>'confidence')::numeric,0.5),
    j->>'summary', j->>'reasoning',
    COALESCE(j->'recommended_actions','[]'::jsonb),
    COALESCE(ARRAY(SELECT jsonb_array_elements_text(j->'mitre_techniques')), ARRAY[]::text[]),
    j->>'model', j->>'prompt_version', COALESCE((j->>'latency_ms')::int,0),
    COALESCE(NULLIF(j->>'status',''),'completed')::triage_status,
    COALESCE((j->>'needs_human')::boolean,false),
    COALESCE((j->>'fallback_used')::boolean,false),
    COALESCE(j->'raw_response','{}'::jsonb), now()
```

```
  FROM payload
  RETURNING alert_id, verdict, severity
)
UPDATE alerts a
  SET status = CASE
    WHEN ins.verdict = 'false_positive' THEN 'false_positive'::alert_status
    WHEN ins.verdict = 'true_positive' AND ins.severity IN ('critical','high')
      THEN 'escalated'::alert_status
    ELSE 'triaged'::alert_status
  END
 FROM ins
 WHERE a.id = ins.alert_id
RETURNING a.id, a.status;
```

Nodo 11 · ¿Notificar? (IF)

- **Condición (OR):** severidad critical o high.
- **True:** Slack → osTicket → Auditoría.
- **False:** Auditoría directa (media/baja se registran, no se notifican).

Nodo 12 · Slack

- **Método:** POST al webhook de Slack ($env.SLACK_WEBHOOK_URL).
- **Mensaje:** severidad, resumen del LLM, veredicto y activo.
- **Fallo:** onError=continueRegularOutput; una caída de Slack no bloquea el pipeline.

Nodo 13 · osTicket / Redmine

- **Método:** POST {OSTICKET_URL}/api/v1/tickets.
- **Cabecera:** Authorization: Bearer $OSTICKET_TOKEN.
- **Body:** asunto, alert_id, prioridad y resumen.
- **Fallo:** continuar; el incidente local ya existe y puede reintentarse.

Nodo 14 · Auditoría

- **Método:** POST {SOCLITE_API_URL}/api/v1/audit/events.
- **Cabecera:** X-Ingest-Key.
- **Body:** module=triage, action=TRIAGE_COMPLETED, entity_id=alert_id, veredicto, severidad, estado y workflow.
- **Efecto:** inserta en audit_log con hash chain.

Nodo 15 · Responder OK

```
{"status": "processed", "alert_id": "...", "verdict": "true_positive", "severity": "critical"}
```

Ruta de error

45. **Marcar pendiente:** UPDATE alerts SET status='pending' WHERE id=$1::uuid RETURNING id,status.
46. **Auditoria error:** action=TRIAGE_FAILED, result=failure, motivo `microservicio LangChain no disponible`.
47. **Responder error:** 202 {status:"pending", reason:"triaje no disponible"}.

Así el SIEM no reintenta indefinidamente y el analista ve la alerta en la cola.

5.5 Manejo de errores, reintentos e idempotencia

Situación	Comportamiento
Payload inválido	Code node lanza error; n8n registra la ejecución fallida (revisar manualmente)
Severidad baja	Se descarta con 202 y no se persiste
Duplicado en 15 min	dedup_count++, 202 duplicate, sin LLM
LangChain timeout/500	2 reintentos → pending + auditoría de fallo
Slack/osTicket caídos	onError=continueRegularOutput; se conserva el triaje
PostgreSQL caído	Nodo falla; n8n reintenta según configuración global; alerta se recupera al reprocesar
Ejecución duplicada	fingerprint único + ON CONFLICT; el triaje no se duplica si se detecta duplicado
Webhook repetido por el SIEM	Misma huella → duplicado

Recomendación: configurar un workflow de error global de n8n (Error Trigger) que envíe a Slack/auditoría las ejecuciones fallidas y permita reprocesar el executionId. No se incluye en este JSON para no acoplar credenciales; el flujo principal ya cubre el fallo de triaje.

5.6 Instalación y activación

48. **Importar:** n8n → *Workflows* → *Import from File* → n8n/soc_lite_triage_workflow.json.
49. **Credencial PostgreSQL:** crear SOC-Lite PostgreSQL con host db, puerto 5432, base soclite, usuario/clave del .env. Asignarla a **Persistir alerta**, **Guardar triaje** y **Marcar pendiente**.
50. **Variables de entorno del contenedor n8n:**

```
LANGCHAIN_URL=http://triage:8001
SOCLITE_API_URL=http://backend:8000
SOCLITE_INGEST_KEY=<misma que INGEST_API_KEY>
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/...
OSTICKET_URL=https://osticket.local
OSTICKET_TOKEN=...
```

51. **Activar** el workflow (toggle *Active*).
52. **URL del webhook:** http://<host>:5679/webhook/soc-lite-ingest.
53. **Probar:**

```
curl -i -X POST http://localhost:5679/webhook/soc-lite-ingest \
  -H 'Content-Type: application/json' \
  -d '{"id":"n8n-demo-1","rule":{"id":"5710","level":12,"description":"Fuerza bruta SSH"},
      "agent":{"name":"SRV-WEB01"},"data":{"srcip":"203.0.113.44","dstip":"10.0.4.21"}}'
```

Respuesta esperada: 202 con {"status":"processed","verdict":"...","severity":"..."}.

Configurar Wazuh

En el manager, una integración custom-* o un <integration> que ejecute un script/HTTP hacia el webhook. Alternativa sin tocar Wazuh: usar el endpoint de ingesta de FastAPI (POST /api/v1/alerts/ingest con X-Ingest-Key) y dejar que n8n consuma la cola por polling.

5.7 Operación y diagnóstico

Síntoma	Causa probable	Acción
202 duplicate constante	Ventana de dedup muy amplia	Ajustar fingerprint/ventana o excluir external_id
pending tras cada alerta	LangChain caído	Revisar docker logs soclite-triage, /health
No llega a Slack	Webhook inválido o onError silencioso	Revisar ejecución y SLACK_WEBHOOK_URL
Error de credencial PostgreSQL	Credencial no asignada	Asignar en los 3 nodos
SQL error en Guardar triaje	Esquema desactualizado	Aplicar db/schema.sql
Respuesta vacía al SIEM	responseMode mal configurado	Debe ser responseNode y existir un Respond

Reprocesar: en n8n, abrir la ejecución fallida → *Retry* (si se guardó el payload) o reenviar el JSON desde el SIEM. La idempotencia evita duplicados.

5.8 Validación realizada

Prueba	Resultado
Importación en n8nio/n8n:latest	✅ 21 nodos importados
Exportación posterior y comparación de tipos	✅ Webhook v2, IF v2.2, Switch v3, Postgres v2.4, HTTP v4.2
INSERT de alerta contra PostgreSQL real	✅ devuelve id, severity, deduplicated
INSERT de triaje + UPDATE de alerta	✅ devuelve id, status
Ejecución E2E con credenciales	⚠️ No ejecutada en este entorno (requiere crear credencial y variables)

La limitación es de **configuración**, no de diseño: el workflow importa y el SQL es válido. Para una demo completa, seguir §5.6 y verificar la respuesta del webhook.

6. Datasets públicos recomendados

> Para entrenar, evaluar y demostrar SOC-Lite sin usar datos reales de la organización.
>
> **Verificar siempre la licencia y los términos de uso antes de incorporarlos a un producto.**

6.1 Cómo encajan en la arquitectura

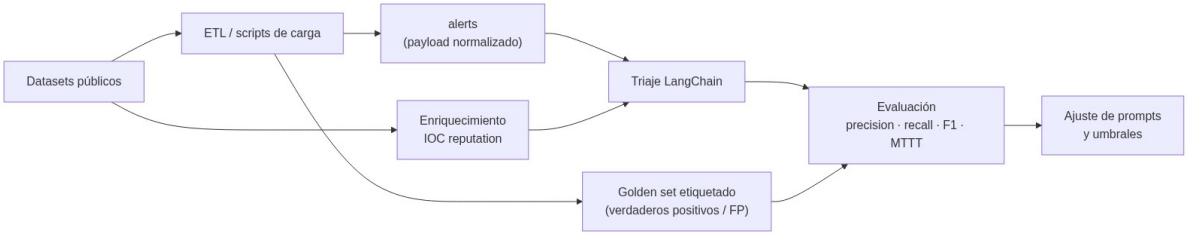


Figura 10. Diagrama de arquitectura / flujo

SOC-Lite necesita tres tipos de datos:

54. **Datos de ingesta** (logs/alertas) para probar el pipeline y el dashboard.
55. **Datos etiquetados** (malicioso/benigno) para medir la calidad del triaje.

56. **Datos de reputación** (IPs, dominios, hashes) para enriquecer y simular proveedores.

6.2 Datasets de red e intrusiones

Dataset	Contenido	Formato / tamaño	Acceso y licencia	Uso en SOC-Lite	Prioridad
CICIDS2017 — Canadian Institute for Cybersecurity	Tráfico benigno + ataques (DoS, brute force, web, infiltración, botnet), con etiquetas por flujo	CSV/PCAP, ~3 M flujos	unb.ca/cic/datasets/ids-2017.html · formulario, uso académico	Generar alertas de red y evaluar detección; mapear a reglas Wazuh/Suricata	Muy alta
CSE-CIC-IDS2018	Similar a 2017, con ataques más variados y entorno AWS	CSV/PCAP, ~16 M flujos	unb.ca/cic/datasets/ids-2018.html · AWS Open Data	Pruebas de volumen y MTTT	Alta
NSL-KDD	Versión depurada de KDD'99, 4 categorías de ataque	CSV, ~150 k registros	unb.ca/cic/datasets/nsl.html · uso académico	Baseline clásico y comparación	Media
UNSW-NB15	9 tipos de ataque modernos + tráfico normal, features de flujo	CSV/PCAP, ~2.5 M	research.unsw.edu.au/projects/unsw-nb15-dataset · académico	Evaluación de clasificadores y reglas	Alta
AIT Log Data Set v2	Logs reales de un entorno empresarial con ataques etiquetados (Linux/Windows)	JSON/CSV, ~1.5 M eventos	ait.ac.at · solicitud, investigación	Ingesta tipo Wazuh y evaluación de correlación	Muy alta
CIC IoT Dataset 2023	Tráfico IoT con 33 ataques	CSV/PCAP	unb.ca/cic/datasets/iotdataset-2023.html · académico	Casos de IoT para PYME	Media
MAWILab	Trazas de backbone con anomalías etiquetadas	PCAP/CSV	fukuda-lab.org/mawilab · abierto	Detección de anomalías de red	Baja
Kyoto 2006+	Honeypot + tráfico normal etiquetado	CSV	takakura.com/Kyoto_data · abierto	Experimentos de FP	Baja

6.3 Datos de endpoint, SIEM y SOC

Dataset	Contenido	Formato	Acceso y licencia	Uso en SOC-Lite	Prioridad
Mordor — OTRF	Eventos de endpoint/Sysmon, PowerShell, cmd, red, con escenarios ATT&CK	JSON (EVTX convertido), GB	github.com/OTRF/mordor · Apache-2.0	Simular alertas de endpoint y MITRE	Muy alta
Security-Datasets — OTRF	Colección curada de datasets de seguridad con metadatos ATT&CK	JSON/EVTX	github.com/OTRF/Security-Datasets · Apache-2.0	Reproducir ataques y evaluar mapeo MITRE	Muy alta

Dataset	Contenido	Formato	Acceso y licencia	Uso en SOC-Lite	Prioridad
Splunk Attack Data	Datos de ataque reproducibles, mapeados a ATT&CK, múltiples fuentes	JSON/EVTX/PCAP	github.com/splunk/attack_data · Apache-2.0	Ingesta masiva y pruebas de regresión	Muy alta
EVTX-ATTACK-SAMPLES	Muestras EVTX de técnicas ATT&CK	EVTX	github.com/sbousseaden/EVTX-ATTACK-SAMPLES · verificar licencia	Pruebas de parsing y detección Windows	Alta
BOTS v1/v2/v3 — Splunk	Datasets de "Boss of the SOC" con logs de múltiples fuentes	JSON/CSV, GB	github.com/splunk/botsv1 y variantes · uso educativo	Simulación SOC completa, hunting y triaje	Muy alta
LogHub	19+ datasets de logs (HDFS, Linux, Apache, Windows, etc.)	CSV/JSON	github.com/logpai/loghub · investigación	Pruebas de parsing y volumen	Media
Wazuh ruleset + samples	Reglas y ejemplos de alertas	XML/JSON	github.com/wazuh/wazuh · GPL-2.0	Validar el normalizador y niveles	Alta
Atomic Red Team	Pruebas atómicas por técnica ATT&CK	YAML/scripts	github.com/redcanaryco/atomic-red-team · MIT	Generar telemetría propia en laboratorio	Alta

BOTS es especialmente útil para SOC-Lite: incluye datos de servidores web, firewalls, endpoints y correo, y escenarios de investigación que se parecen a la operación real de una PYME.

6.4 Threat intelligence y reputación

Fuente	Contenido	Acceso / API	Uso en SOC-Lite	Licencia
VirusTotal	Reputación de IP/dominio/hash/URL	API v3 (clave gratuita limitada)	Tool `virustotal` del agente	Términos de VirusTotal; no redistribuir
AbuseIPDB	Reportes de abuso por IP, score 0–100	API v2 (clave gratuita)	Tool `abuseipdb`	Términos; atribución
AlienVault OTX	Indicadores y pulsos comunitarios	API / descargas	Enriquecimiento y datasets de IOC	Apache-2.0 (verificar por pulso)
URLhaus (abuse.ch)	URLs de distribución de malware	CSV/JSON diario	Enriquecimiento de URLs/dominios	CC0 / términos abuse.ch
MalwareBazaar (abuse.ch)	Muestras y hashes de malware	API/CSV	Hash reputation, laboratorio aislado	Términos abuse.ch
Feodo Tracker (abuse.ch)	IPs de C2 de botnets	CSV/JSON	Bloqueo y detección	CC0
PhishTank	URLs de phishing verificadas	CSV/API	Casos de phishing	Términos, atribución
MITRE ATT&CK	Técnicas, tácticas, grupos, software	STIX/JSON	Mapeo de técnicas y reportes	Términos MITRE, uso libre con atribución
CISA KEV	Vulnerabilidades explotadas conocidas	JSON/CSV	Priorización de vulnerabilidades (caso futuro)	Dominio público
NVD / CVE	Vulnerabilidades y CVSS	API/JSON	Módulo de vulnerabilidades	NIST, uso libre

Regla de oro: no envíes el payload completo a estos proveedores; SOC-Lite sólo consulta el indicador (IP/dominio/hash). Las claves se leen de variables de entorno y nunca se almacenan en la base.

6.5 Datasets para evaluar el LLM

No existe un dataset público específico de "triaje SOC en español" con calidad suficiente. Se recomienda **construir un golden set propio** a partir de los anteriores:

Estrategia	Descripción	Esfuerzo
Muestreo estratificado de BOTS/Mordor	200–500 alertas con etiqueta humana (TP/FP/sospechoso/benigno)	Medio
Inyección de casos sintéticos	Variantes de las 8 historias curadas, con cambios de IP/activo/descripción	Bajo
Adjudicación doble	Dos analistas etiquetan a ciegas; un tercero resuelve discrepancias	Alto
Datos de la organización	Alertas históricas ya cerradas, anonimizadas	Alto (privacidad)

Formato sugerido (JSONL):

```
{"alert_id":"gold-001","source":"wazuh","rule_level":12,"description":"Fuerza bruta SSH...",
 "src_ip":"203.0.113.44","asset_criticality":5,
 "expected_verdict":"true_positive","expected_severity":"critical",
 "expected_mitre":["T1110"],"notes":"IP con 100% AbuseIPDB"}
```

Métricas:

Métrica	Fórmula	Objetivo inicial
Precision (TP)	TP / (TP + FP)	> 0.80
Recall (TP)	TP / (TP + FN)	> 0.85
F1	2PR / (P + R)	> 0.82
Tasa de FP	FP / total	< 15 %
Cobertura	alertas con veredicto / total	> 95 %
MTTT	mediana y P95	< 30 / 60 min
Acuerdo con humano	Cohen's kappa	> 0.70

Advertencia: usar un LLM como juez introduce sesgo. La evaluación final debe ser humana y ciega al modelo; el LLM sólo sirve para pre-etiquetar y detectar casos raros.

6.6 Cómo cargar los datasets en SOC-Lite

6.6.1 Alertas normalizadas

Los datasets de red vienen como flujos, no como alertas. El pipeline recomendado es:

```
PCAP/CSV → Suricata/Zeek o script ETL → JSON con forma Wazuh → POST /api/v1/alerts/ingest
```

Script de ejemplo (Python + pandas) para convertir CICIDS a alertas:

```
import pandas as pd, requests, uuid

API = "http://localhost:8010/api/v1/alerts/ingest"
KEY = "dev-ingest-key-change-me"

df = pd.read_csv("cicids2017.csv").sample(500, random_state=42)
```

```
for _, row in df.iterrows():
    payload = {
        "id": f"cic-{uuid.uuid4().hex[:8]}",
        "source": "suricata",
        "rule": {"id": "CIC-001", "level": 10 if row["Label"] != "BENIGN" else 3,
                 "description": f"Flujo {row['Label']} hacia {row['Destination IP']}"},
        "agent": {"name": "LAB-SENSOR", "criticality": 3},
        "data": {"srcip": row["Source IP"], "dstip": row["Destination IP"],
                 "dstport": int(row["Destination Port"]), "protocol": row["Protocol"].lower()},
    }
    requests.post(API, json=payload, headers={"X-Ingest-Key": KEY}, timeout=10)
```

Anonimizar IPs reales con rangos de documentación (RFC 5737: 192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24) antes de cargar datos de producción.

6.6.2 Evaluación del triaje

57. Cargar 200–500 alertas del golden set con external_id único.
58. Ejecutar POST /alerts/{id}/triage para cada una (script con concurrencia limitada).
59. Comparar triage_results.verdict / severity contra la etiqueta.
60. Calcular la matriz de confusión y las métricas; registrar model y prompt_version.
61. Repetir al cambiar de modelo o prompt; comparar.

6.6.3 Enriquecimiento simulados

Para pruebas sin claves de API, usar los CSV de URLhaus/Feodo como un "proveedor local":

```
# pseudo-tool para tests
def lookup_local(ip: str) -> dict:
    row = feodo_df[feodo_df["dst_ip"] == ip]
    return {"provider": "feodo", "verdict": "malicious" if len(row) else "unknown", "score": 90}
```

SOC-Lite ya soporta provider=internal en enrichment_data; se puede cargar una tabla de IOCs conocidos y consultarla antes de salir a Internet.

6.7 Licencias y privacidad

Aspecto	Recomendación
Uso comercial	CIC/UNSW/AIT suelen ser sólo investigación; solicitar permiso explícito
Atribución	Citar autores y papers en la documentación y en la UI si se muestran resultados
Redistribución	No redistribuir PCAP/EVTX con datos personales; publicar sólo el script de descarga
Datos personales	Anonimizar IPs, usuarios, dominios y hashes de archivos personales
Términos de API	Respetar cuotas de VirusTotal/AbuseIPDB; cachear con TTL
Modelos	Verificar la licencia del modelo Ollama (Llama 3.1 tiene su propia licencia comunitaria)
Retención	No conservar más de lo necesario; TTL en enrichment_data

---

6.8 Prioridad de descarga recomendada

Orden	Dataset	Por qué
1	**BOTS v1**	Logs multi-fuente y escenarios SOC listos para triaje
2	**Mordor / Security-Datasets**	Endpoint + ATT&CK, excelente para el agente

Orden	Dataset	Por qué
3	**CICIDS2017**	Volumen y etiquetas de red para probar ingesta y métricas
4	**AIT Log Data Set v2**	Logs empresariales realistas para Wazuh
5	**URLhaus / Feodo / OTX**	Reputación para enriquecimiento y pruebas offline
6	**MITRE ATT&CK STIX**	Mapeo de técnicas y reportes
7	**Splunk Attack Data**	Regresión amplia cuando el pipeline ya funciona

Con **BOTS + Mordor + CICIDS** se puede demostrar el 90 % de las capacidades de SOC-Lite sin tocar datos reales. El 10 % restante (política, activos internos, contexto del negocio) requiere datos de la organización y un proceso de anonimización.

7. Stack tecnológico detallado

> Criterio: **mínimo coste operativo, máxima trazabilidad y soberanía del dato**. Todo el software es open source y desplegable on-premise; no hay dependencia obligatoria de la nube.

7.1 Resumen

Capa	Tecnología	Versión fijada	Licencia	Por qué
Lenguaje backend	Python	3.11	PSF	Ecosistema IA, tipado moderno, soporte LTS de librerías
Framework API	FastAPI	0.115.6	MIT	Async nativo, Pydantic, OpenAPI automático
ORM	SQLAlchemy	2.0.36	MIT	ORM maduro con soporte async real y tipado
Driver	asyncpg	0.30.0	Apache-2.0	Rendimiento y pool para PostgreSQL
Validación	Pydantic	2.10.4	MIT	Contratos estrictos y serialización
Auth	PyJWT + bcrypt	2.10.1 / 4.2.1	MIT / Apache-2.0	JWT estándar y hashing robusto
HTTP cliente	httpx	0.28.1	BSD-3	Cliente async con timeouts
PDF	ReportLab	4.2.5	BSD	Reportes reproducibles en servidor
Word	python-docx	1.1.2	MIT	DOCX programático
Excel	openpyxl	3.1.5	MIT	XLSX con estilos y gráficos nativos
Base de datos	PostgreSQL	16	PostgreSQL License	JSONB, INET, enums, percentiles, integridad
Orquestación	n8n	latest (1.x)	Sustainable Use License*	Integraciones, reintentos y visibilidad
IA	LangChain Core + langchain-ollama	≥1.0, <2	MIT	Tool calling, prompts, cadenas
LLM local	Ollama (llama3.1:8b / mistral)	latest	MIT (Ollama) / licencia del modelo	Soberanía del dato, coste marginal cero
Frontend	React + TypeScript	18.3.1 / 5.6.3	MIT / Apache-2.0	Ecosistema, tipado, componentes

Capa	Tecnología	Versión fijada	Licencia	Por qué
Build	Vite	5.4.11	MIT	Dev rápido, build optimizado
UI	Tailwind CSS	3.4.17	MIT	Diseño consistente sin CSS muerto
Datos remotos	TanStack Query	5.62.7	MIT	Caché, reintentos, invalidación
Gráficos	Recharts	2.15.0	MIT	Charts declarativos para React
Ruteo	React Router	6.28.0	MIT	Rutas protegidas
HTTP frontend	Axios	1.7.9	MIT	Interceptores de JWT/refresh
Servidor web	Nginx	1.27-alpine	BSD-2	SPA, proxy, TLS, CSP
Contenedores	Docker + Compose	29.x / v2	Apache-2.0	Reproducibilidad y despliegue
Pruebas	pytest + pytest-asyncio	8.3.4 / 0.25	MIT / Apache-2.0	Integración async contra BD real

* n8n es "fair-code": uso interno y autoalojado permitido; revisar la licencia si se ofrece como SaaS a terceros.

7.2 Backend

¿Por qué FastAPI y no Django/Flask?

Criterio	FastAPI	Django REST	Flask
Async nativo	✅	⚠️ parcial (ASGI desde 4.1)	⚠️ requiere extensiones
Validación + OpenAPI	✅ Pydantic + Swagger	✅ DRF + drf-spectacular	❌ manual
Curva de aprendizaje	Baja	Media	Baja
Admin incluido	❌	✅	❌
Ideal para LLM/streaming	✅	⚠️	✅
Ecosistema ORM	SQLAlchemy 2	Django ORM	SQLAlchemy

Decisión: FastAPI por async, contratos Pydantic y documentación automática. No se necesita el admin de Django porque la consola React cubre la operación.

¿Por qué SQLAlchemy 2 async?

- Mismo modelo para lectura y escritura, con selectinload para evitar N+1.
- Compatible con Alembic para migraciones futuras.
- asyncpg entrega ~2–3× más throughput que psycopg2 en cargas de I/O.

¿Por qué no un ORM "mágico" o SQL plano?

- SQL plano en servicios críticos (dashboard, reportes) para aprovechar percentile_cont, FILTER, CTEs y LATERAL.
- ORM para CRUD y relaciones, evitando SQL injection y manteniendo tipos.

7.3 Frontend

¿Por qué React + TypeScript?

- Tipado estricto contra el contrato de la API (types/index.ts), lo que detectó errores en build.
- Ecosistema de tablas/gráficos y familiaridad del equipo.
- Vite reduce el ciclo de feedback (HMR) y produce chunks pequeños.

¿Por qué Tailwind y no una librería de componentes?

- Control total del diseño oscuro y de los estados de severidad.
- Sin dependencia de un tema propietario; el CSS final es pequeño (~34 kB).
- Componentes propios reutilizables (.card, .btn, .table).

¿Por qué TanStack Query?

- El dashboard tiene 7 consultas independientes que se pueden cachear e invalidar por separado.
- Evita duplicar estado global y maneja reintentos/cancelación.
- placeholderData mantiene la tabla estable al paginar.

¿Por qué Recharts?

- Declarativo, responsive y suficiente para área/dona/barras/líneas.
- Alternativas: ECharts (más potente, más peso), Chart.js (imperativo), D3 (curva alta).
- El chunk de Recharts (~423 kB) se separa y cachea; sólo se descarga al entrar al dashboard.

7.4 Datos

PostgreSQL 16

Necesidad	Ventaja de PostgreSQL
Payload heterogéneo	JSONB indexable con GIN
IPs	tipo INET con operadores de red
Métricas	percentile_cont, FILTER, generate_series, CTEs
Integridad	FKs, CHECK, enums nativos, triggers
Concurrencia	MVCC, advisory locks para la cadena de auditoría
Extensiones	pgcrypto (hash), pg_trgm (búsqueda), citext

¿Por qué no MongoDB? Las métricas y los joins (alerta → triaje → incidente → usuario) son relacionales; MongoDB obligaría a duplicar datos y perdería integridad referencial.

¿Por qué no Elasticsearch? Excelente para búsqueda de logs a gran escala, pero añade un clúster y operación. Para 5 000 alertas/día, PostgreSQL con índices GIN/trigram es suficiente. Se puede añadir Elasticsearch como capa de búsqueda si el volumen crece.

¿Por qué no MySQL? JSONB, INET, enums y funciones de percentiles son más limitados; la migración no aporta ventaja.

Volumen y crecimiento

Escenario	Alertas/día	Almacenamiento anual estimado	Recomendación
PYME	500–2 000	10–40 GB	1 instancia, 4 vCPU/8 GB
Mediana	5 000–20 000	50–200 GB	8 vCPU/32 GB + réplica de lectura

Escenario	Alertas/día	Almacenamiento anual estimado	Recomendación
Pública	20 000–100 000	200 GB–1 TB	Particionado + archivado + HA

7.5 IA y orquestación

LangChain vs LangFlow

Criterio	LangChain (elegido)	LangFlow
Tool calling y agentes	✅ control total	⚠️ visual, limitado
Versionado de prompts	✅ en código	⚠️ en el grafo
Tests unitarios	✅	❌
Memoria de casos	✅	⚠️
Prototipado rápido	⚠️	✅
Perfil del equipo	Desarrolladores	Analistas

Decisión: LangChain en un microservicio FastAPI. LangFlow se puede usar para explorar prompts, pero no para producción con validación estricta.

n8n vs alternativas

Criterio	n8n (elegido)	Airflow	Temporal	Python puro
Webhooks event-driven	✅	⚠️	✅	✅
Integraciones listas	✅ 400+	⚠️	❌	❌
Reintentos/visibilidad	✅	✅	✅	❌
Operación por analistas	✅	❌	❌	❌
Complejidad operativa	Baja	Media	Alta	Baja

Decisión: n8n por velocidad de integración y autonomía del equipo SOC. La lógica crítica se mantiene en Python testeable.

Ollama vs OpenAI

Criterio	Ollama local (default)	OpenAI (opcional)
Privacidad	✅ los datos no salen	❌ payload a terceros
Coste	HW + electricidad	Por token
Calidad	Buena en 8B; menor en razonamiento complejo	Superior
Latencia	5–25 s en GPU/CPU moderada	1–5 s
Disponibilidad	Depende del host	SLA del proveedor
Cumplimiento	Soberanía	Requiere DPA y evaluación

Decisión: Ollama por defecto; el proveedor es una variable de entorno. La arquitectura permite cambiar sin tocar n8n ni el frontend.

7.6 Infraestructura

Componente	Tecnología	Notas
Contenedores	Docker Engine 24+ / Compose v2	Red interna, volúmenes nombrados
Proxy/TLS	Nginx 1.27	Terminación TLS, CSP, rate limiting (futuro)
Orquestación	Docker Compose	Suficiente para una instancia; Kubernetes si HA
Backups	`pg_dump` + WAL	Diario + retención 30 días
Observabilidad	Logs JSON + `/health`	Pendiente: Prometheus/Grafana/Loki
Secretos	`.env` + gestor externo	Vault/SOPS en producción

Puertos

Servicio	Host	Contenedor
Frontend	8090	80
API	8010	8000
Triaje	8011	8001
n8n	5679	5678
PostgreSQL	5439	5432
Ollama	11435	11434

7.7 Dimensionamiento y coste

Perfiles

Perfil	Endpoints	Alertas/día	HW recomendado	LLM	Coste mensual estimado*
PYME	50–200	500–2 000	4 vCPU, 8 GB, 80 GB	Ollama 8B en CPU o API	USD 40–120
Mediana	200–1 000	5 000–20 000	8 vCPU, 32 GB, 500 GB	Ollama 8B + GPU T4 opcional	USD 250–600
Pública	1 000+	20 000+	2 nodos, 16 vCPU, 64 GB, 1 TB	Ollama 70B o API privada	USD 800–2 500

* Sólo infraestructura/energía; el software es open source. Los precios de nube varían por región y proveedor. No incluye horas de operación ni auditoría.

Comparación con SOAR comercial

Concepto	SOAR comercial	SOC-Lite
Licencia anual	USD 30 000–150 000	0
Implementación	3–6 meses	1–2 semanas
Personal dedicado	1–2 FTE	0.2–0.5 FTE
Coste LLM	Aparte	Local (0) o API (~USD 10–50/mes)
Lock-in	Alto	Bajo (todo open source)

La comparación es orientativa: un SOAR comercial ofrece soporte 24/7, conectores certificados y

cumplimiento contractual que SOC-Lite no incluye.

7.8 Requisitos no funcionales

Requisito	Cómo se cumple	Evidencia
Usabilidad	UI oscura, filtros, atajos visuales, feedback inmediato	Capturas y build
Rendimiento	Async, índices, paginación, chunks	14 tests; dashboard < 300 ms en 728 alertas
Confiabilidad	Fallback heurístico, healthchecks, reintentos n8n	Pruebas de triaje degradado
Seguridad	JWT, RBAC, bcrypt, CSP, auditoría	Tests de RBAC y cadena
Interoperabilidad	API REST + OpenAPI + webhook estándar	/docs, contrato
Escalabilidad	API sin estado, pool, Docker	N réplicas; particionado futuro
Mantenibilidad	Tipado estricto, tests, servicios, docs	tsc 0 errores, 14 tests
Portabilidad	Docker Compose, variables de entorno	docker compose up

7.9 Riesgos del stack y mitigaciones

Riesgo	Impacto	Mitigación
Ollama no soporta tool calling en algún modelo	Triaje degradado	Fallback heurístico + elección de modelo validada
n8n cambia su licencia	Coste legal	La lógica crítica está en Python; n8n es reemplazable
Recharts pesa 423 kB	Carga inicial del dashboard	Chunk separado y cacheado; lazy load si crece
LangChain 1.x introdujo cambios	Ruptura de imports	Versiones pinneadas y tests de contrato
PostgreSQL sin HA	Caída del servicio	Backup + réplica; HA en perfil público
Dependencia de APIs externas	Enriquecimiento incompleto	skipped explícito, caché, proveedores locales
Complejidad de credenciales n8n	Fricción de despliegue	Documentado paso a paso; variables de entorno

7.10 Política de versiones y actualización

- **Backend:** versiones exactas en requirements.txt; actualizar en rama con tests.
- **Frontend:** rangos ^ en package.json + package-lock.json; CI con npm ci.
- **Base de datos:** migraciones Alembic a partir de la v1.1; los cambios de enum se prueban en staging.
- **Modelos:** OLLAMA_MODEL es configurable; cada cambio se evalúa con el golden set y se registra model/prompt_version en cada triaje.
- **n8n:** exportar el workflow a Git tras cada cambio; importar en staging antes de producción.

7.11 Conclusión

El stack no busca novedad, sino **adecuación al problema y al equipo**:

- Python/FastAPI/LangChain porque el triaje exige IA, validación y tests.
- PostgreSQL porque las métricas y la auditoría son relacionales e íntegras.
- n8n porque la operación necesita integraciones visibles y reintentos sin desplegar código.
- React/TypeScript porque la consola debe ser rápida, accesible y mantenible.
- Ollama porque la privacidad del incidente no es negociable en PYME y sector público.

El resultado es un sistema de ~10 500 líneas de código (más SQL, workflow y documentación) que se levanta con `docker compose up`, se opera desde el navegador y deja evidencia verificable de cada acción.

Contrato de API v1

> Base: `http://localhost:8010/api/v1` · OpenAPI interactivo: `/docs` · Redoc: `/redoc`
>
> Autenticación: `Authorization: Bearer <access_token>` salvo login/refresh e ingesta.

Convenciones

Tema	Convención
Formato	JSON UTF-8
Fechas	ISO 8601 con zona (`2026-10-20T10:15:51.792Z`)
Paginación	`page` (1..N), `size` (1..200); respuesta `{items,total,page,size,pages}`
Errores	`{detail, code, request_id, path}`; validación añade `errors[]`
IDs	UUID v4
Orden	`sort=received_at
Idempotencia	Ingesta por `fingerprint`; triaje por ventana de 60 s o `force=true`

Autenticación

POST /auth/login

```
{"username": "analista1", "password": "Analista123!"}
{
 "access_token": "...",
 "refresh_token": "...",
 "token_type": "bearer",
 "expires_in": 1800,
 "user": {
  "id": "...", "username": "analista1", "email": "analista1@soclite.example.com",
  "full_name": "Carlos Mendoza",
  "role": {"id": 3, "code": "analyst", "name": "Analista SOC"},
  "is_active": true, "mfa_enabled": false,
  "last_login_at": "2026-10-20T10:15:00Z", "created_at": "2026-09-01T08:00:00Z",
  "permissions": ["alerts:read", "alerts:create", "triage:run", "..."]
 }
}
```

Código	Significado
200	Sesión iniciada
401	Credenciales inválidas o cuenta inactiva
422	Payload inválido

POST /auth/token

application/x-www-form-urlencoded (username, password) para el flujo OAuth2 password. Misma respuesta que /login.

POST /auth/refresh

```
{"refresh_token": "..."}
```

Rota el par. Si el token ya fue rotado, revoca todas las sesiones y devuelve 401.

POST /auth/logout

Requiere access token. Revoca refresh tokens (todos o el indicado). 200 {"detail":"Sesión cerrada"}.

GET /auth/me

Devuelve el perfil y los permisos efectivos (recalculados desde la base).

POST /auth/change-password

```
{"current_password": "Analista123!", "new_password": "NuevaClave2026!"}
```

Valida política, revoca sesiones y audita PASSWORD_CHANGE.

Alertas

POST /alerts/ingest (máquina)

Cabecera obligatoria: X-Ingest-Key: <INGEST_API_KEY>. No usa JWT.

```
{
 "id": "wazuh-9001",
 "timestamp": "2026-10-20T10:00:00Z",
 "rule": {"id": "5710", "level": 10, "description": "Fuerza bruta SSH"},
 "agent": {"name": "SRV-WEB01", "criticality": 5},
 "data": {"srcip": "203.0.113.44", "dstip": "10.0.4.21", "dstport": 22, "protocol": "tcp"}
}
{"status": "accepted", "alert_id": "...", "severity": "critical",
 "deduplicated": false, "message": "Alerta registrada en estado pendiente"}
```

Código	Caso
202	Aceptada o duplicada
401	Clave inválida
422	Payload inválido

GET /alerts

Parámetros: page, size, severity (lista separada por comas), status, source, q, date_from, date_to, sort.

```
{
 "items": [{
  "id": "...", "source": "wazuh", "rule_id": "5710", "rule_level": 10,
  "description": "Fuerza bruta SSH...", "src_ip": "203.0.113.44",
  "agent_name": "SRV-WEB01", "asset_criticality": 5,
  "severity": "critical", "status": "triaged",
  "verdict": "true_positive", "confidence": 0.96,
  "summary": "...", "mttt_minutes": 6.0,
  "received_at": "2026-10-20T10:00:00Z", "incident_id": "...", "ticket_id": "OSK-10001"
 }],
 "total": 379, "page": 1, "size": 15, "pages": 26
}
```

GET /alerts/{id}

Devuelve la alerta completa con triage_results[], enrichments[], feedback[], incident, raw_payload, tags, fingerprint y mttt_minutes.

POST /alerts/{id}/triage?force=false

Ejecuta el triaje. Respuesta:

```
{
 "id": "...", "verdict": "true_positive", "severity": "critical", "confidence": 0.96,
 "summary": "...", "reasoning": "...",
 "recommended_actions": [{"action": "Bloquear IP", "priority": 1, "owner": "SOC"}],
 "mitre_techniques": ["T1110"], "model": "llama3.1:8b", "prompt_version": "v1.2.0",
 "latency_ms": 18400, "status": "completed", "needs_human": false,
 "fallback_used": false, "triaged_at": "2026-10-20T10:01:00Z"
}
```

POST /alerts/{id}/feedback

```
{"is_false_positive": true, "comment": "Escaneo bloqueado por firewall"}
```

POST /alerts/{id}/incident

```
{"priority": "P2", "title": "Investigar SSH", "assignee_id": null}
```

Devuelve IncidentOut con ticket_id, status, priority, sla_due_at.

PATCH /alerts/{id}

```
{"status": "in_review", "tags": ["ssh", "revisar"], "asset_criticality": 4}
```

Incidentes

Método	Ruta	Permiso	Descripción
GET	/incidents	incidents:read	Filtros status, priority, assignee_id, q
GET	/incidents/{id}	incidents:read	Detalle
PATCH	/incidents/{id}	incidents:update / incidents:close	Estado, prioridad, responsable, resolución

Cerrar o resolver exige incidents:close; se asigna closed_at.

Dashboard

Método	Ruta	Permiso	Respuesta
GET	/dashboard/summary?days=30	dashboard:read	{period,totals,rates,mttt,deltas,generated_at}
GET	/dashboard/trend?days=30	dashboard:read	[{date,total,critical,high,medium,low,false_positive}]
GET	/dashboard/severity?days=30	dashboard:read	{items:[{severity,count}],total}
GET	/dashboard/top-sources?days=30&by=ip	dashboard:read	[{key,label,count,critical}]
GET	/dashboard/mttt?days=30	dashboard:read	[{severity,count,avg_minutes,p95_minutes,target_minutes,compliance}]

Método	Ruta	Permiso	Respuesta
GET	/dashboard/fp-rate?days=30	dashboard:read	[{date,triaged,false_positive,rate}]
GET	/dashboard/sla?days=30	dashboard:read	[{severity,target_minutes,total,within_sla,compliance,breaches}]
GET	/dashboard/analysts?days=30	dashboard:read	[{username,full_name,actions,triages}]

days ∈ [1, 365]. by ∈ ip | agent | rule.

Reportes

GET /reports/types

```
[{"type":"operational","name":"Operacional","description":"...","formats":"pdf,docx,xlsx"}]
```

POST /reports

```
{"type": "management", "format": "pdf", "date_from": "2026-09-21", "date_to": "2026-10-20"}
{"id":"...","type":"management","format":"pdf","status":"ready",
 "params":{"days":30,"date_from":"2026-09-21","date_to":"2026-10-20"},
 "file_size":284311,"created_at":"...","completed_at":"...","error":null}
```

GET /reports

Historial paginado.

GET /reports/{id}/download

200 con Content-Disposition: attachment y MIME:

application/pdf, ...wordprocessingml.document, ...spreadsheetml.sheet.

Requiere reports:export. Audita REPORT_DOWNLOAD.

Auditoría

GET /audit

Filtros: module, action, username, result (success | failure | denied), q,

date_from, date_to, page, size.

```
{
 "items": [{
   "id": 142, "occurred_at": "2026-10-20T10:15:51Z", "username": "analista1",
   "module": "triage", "action": "TRIAGE_RUN", "entity_type": "alert",
   "entity_id": "...", "result": "success",
   "detail": {"verdict": "true_positive"}, "ip": "10.10.1.30",
   "request_id": "...", "duration_ms": 14380, "hash": "..."
 }],
 "total": 1250, "page": 1, "size": 20, "pages": 63
}
```

GET /audit/modules

[{"module":"alerts","total":812}, ...]

GET /audit/stats?days=30

{total, success, failure, denied, modules[]}

GET /audit/verify-chain

```
{"valid": true, "checked": 1250, "broken_at": null, "message": "Cadena íntegra"}
```

GET /audit/export?format=csv|xlsx

Descarga CSV/XLSX (máx. 50 000 filas). Requiere audit:export. Audita AUDIT_EXPORT.

POST /audit/events (máquina)

Cabecera X-Ingest-Key. Para n8n:

```
{"module":"triage","action":"TRIAGE_COMPLETED","entity_type":"alert","entity_id":"...",
 "result":"success","username":"n8n","detail":{"verdict":"true_positive"}}
{"id": 150, "hash": "...64 hex..."}
```

Usuarios y roles

Método	Ruta	Permiso	Descripción
GET	/users	users:read	Filtros q, role, is_active
POST	/users	users:create	{username,email,full_name,password,role_id,is_active}
GET	/users/{id}	users:read	Detalle
PATCH	/users/{id}	users:update	Email, nombre, rol, estado, MFA
DELETE	/users/{id}	users:delete	Baja lógica; no permite el último admin
POST	/users/{id}/reset-password	users:update	{new_password}; revoca sesiones
GET	/roles	roles:read	Roles con permisos y user_count
GET	/permissions	roles:read	Catálogo
PUT	/roles/{id}/permissions	roles:update	{permission_codes:[...]}

Salud

Método	Ruta	Auth	Respuesta
GET	/health	—	{status,service,version,environment,database,timestamp}
GET	/health/ready	—	{status,database,triage_service}

Matriz de permisos por endpoint (resumen)

Grupo	Lectura	Escritura	Exportación
Alertas	alerts:read	alerts:create/update/delete, triage:run/feedback	alerts:export
Incidentes	incidents:read	incidents:create/update/close	incidents:export

Grupo	Lectura	Escritura	Exportación
Dashboard	dashboard:read	—	dashboard:export
Usuarios	users:read	users:create/update/delete	—
Roles	roles:read	roles:update	—
Auditoría	audit:read	—	audit:export
Reportes	reports:read	reports:create	reports:export

La matriz efectiva se administra en roles × permissions y se puede editar desde la UI (PUT /roles/{id}/permissions), salvo el rol admin, que es inmutable.